

图 概述

- 图的术语
- 图的表示
- 图的遍历

基本术语

❖ $G = (V; E)$

vertex: $n = |V|$

edge|arc: $e = |E|$



❖ 同一条边的两个顶点，彼此邻接 (adjacency)

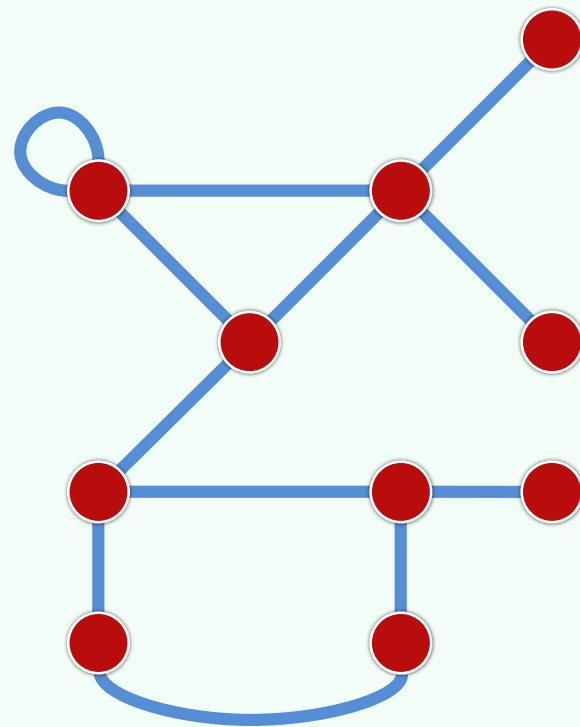
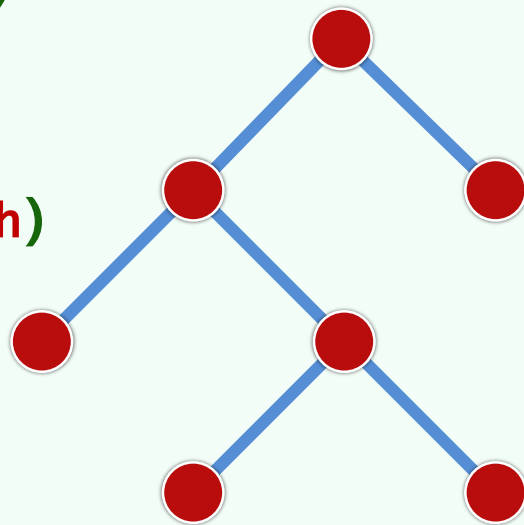
同一顶点自我邻接，构成自环 (self-loop)

不含自环及重边，即为简单图 (simple graph)

非简单 (non-simple) 图，暂不讨论

❖ 顶点与其所属的边，彼此关联 (incidence)

度 (degree/valency) : 与同一顶点关联的边数

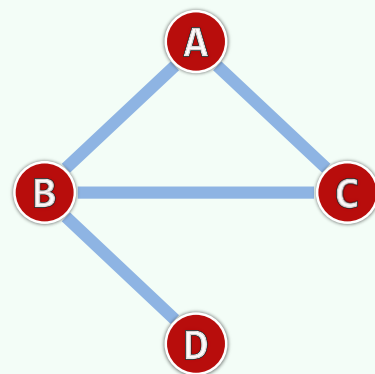


无向图 + 有向图

❖ 若邻接顶点 u 和 v 的次序无所谓

则 (u, v) 为无向边 (undirected edge)

❖ 所有边均无方向的图，即无向图 (undigraph)

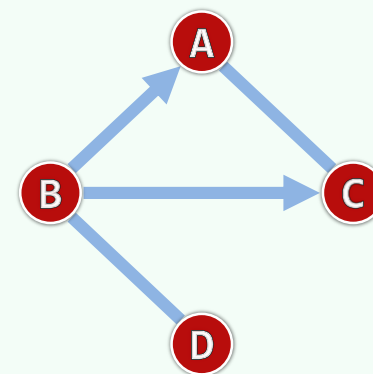


(a) undigraph

❖ 反之，有向图 (digraph) 中均为有向边 (directed edge)

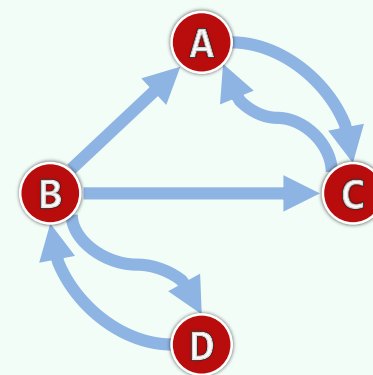
u 、 v 分别称作边 (u, v) 的尾 (tail)、头 (head)

❖ 无向边、有向边并存的图，称作混合图 (mixed graph)



(b) mixed graph

❖ 有向图通用性更强，故本章主要针对有向图介绍相关结构及算法



(c) digraph

路径 + 环路

❖ 路径 $\pi = \langle v_0, v_1, \dots, v_k \rangle$

长度 $|\pi| = k$

❖ 简单路径: $v_i \neq v_j$ 除非 $i = j$

❖ 环/环路: $v_0 = v_k$

❖ 有向无环图 (DAG)

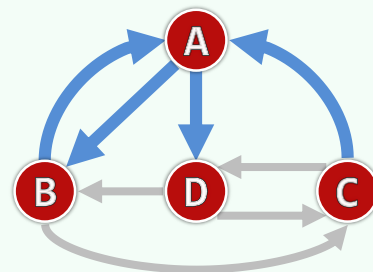
❖ 欧拉环路: $|\pi| = |E|$

各边恰好出现一次

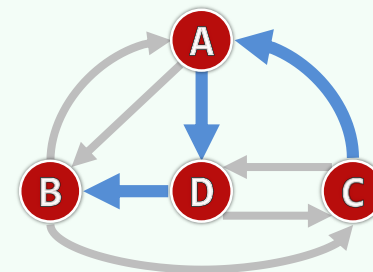
不走回头路

❖ 哈密尔顿环路: $|\pi| = |V|$

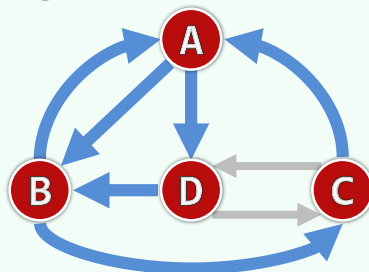
各顶点恰好出现一次



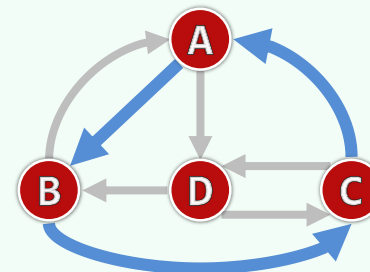
(i) path



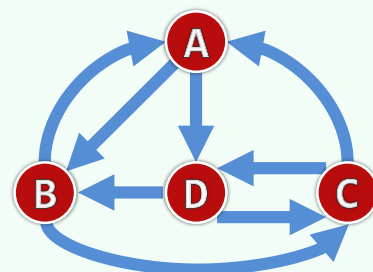
(ii) simple path



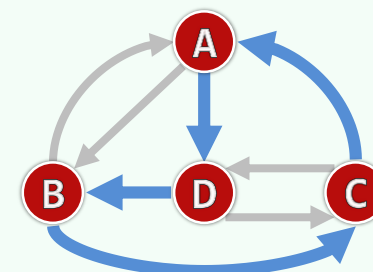
(i) cycle



(ii) simple cycle



(i) Eulerian tour

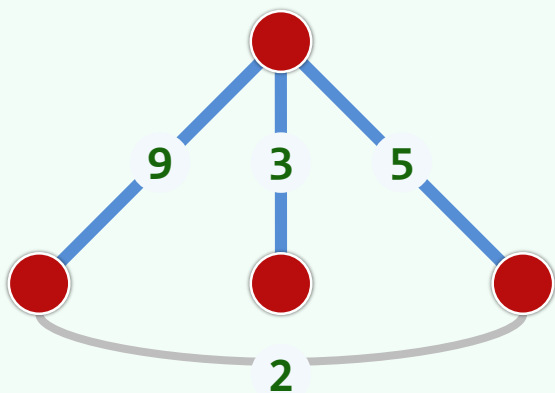


(ii) Hamiltonian tour

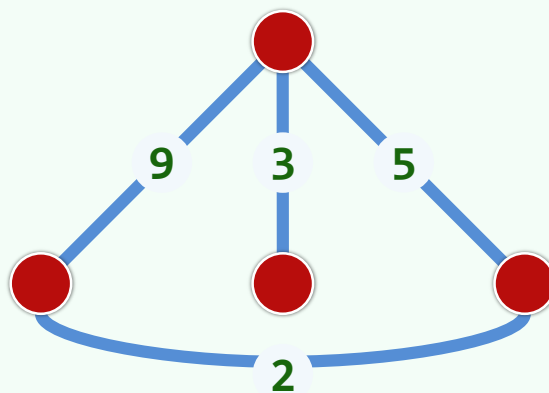
支撑树 + 带权网络 + 最小支撑树

❖ 图 $G = (V; E)$ 的子图 $T = (V; F)$ 若是树，即为其支撑树 (spanning tree)

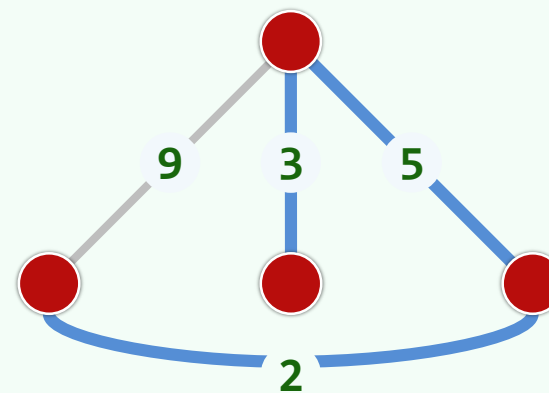
同一图的支撑树，通常并不唯一



spanning tree



weighted network
(triangle inequality?)



minimum spanning tree

❖ 各边 e 均有对应的权值 $wt(e)$ ，则为带权网络 (weighted network)

❖ 同一网络的支撑树中，总权重最小者为最小支撑树 (MST)

图

邻接矩阵：构思

Graph模板类

```
template <typename Tv, typename Te> class Graph {  
private: void reset() { //所有顶点、边的辅助信息复位  
    for ( Rank i = 0; i < n; i++ ) { //顶点  
        status(i) = UNDISCOVERED; dTime(i) = fTime(i) = -1;  
        parent(i) = -1; priority(i) = INT_MAX;  
        for ( Rank j = 0; j < n; j++ ) //边  
            if ( exists(i, j) ) type(i, j) = UNDETERMINED;  
    } //for  
} //reset  
public: int n, e; //顶点、边数目  
    /* ... 顶点操作、边操作、图算法：无论如何实现，接口必须统一 ... */  
} //Graph
```

邻接矩阵 + 关联矩阵

❖ adjacency matrix: 记录**顶点**之间的**邻接**关系

——对应: 矩阵元素 $\Leftrightarrow$ 图中可能存在的边

- $A(i, j) = 1$ (若顶点*i*与*j*之间存在一条边)

- $= 0$ (否则)

既然只考察简单图, 对角线统一设置为0

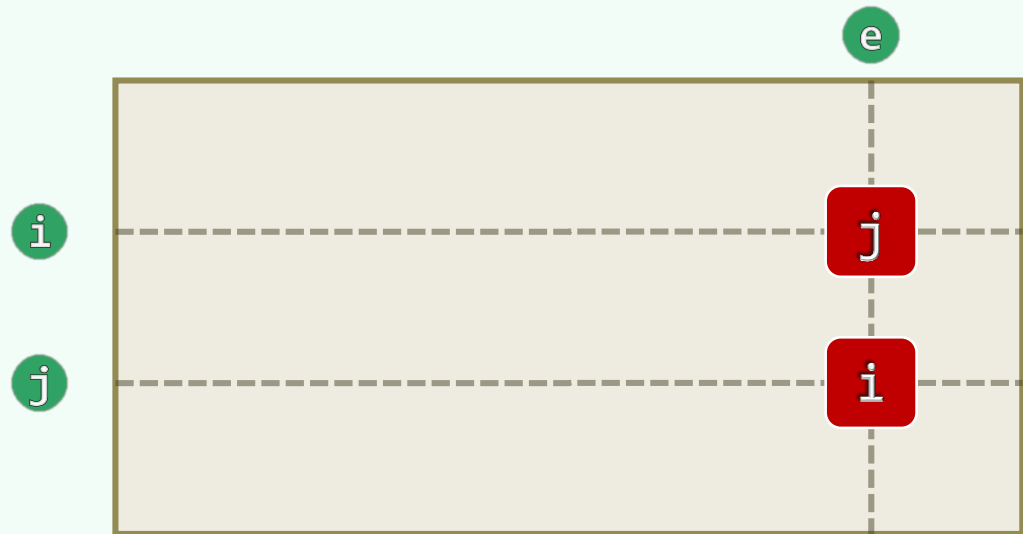
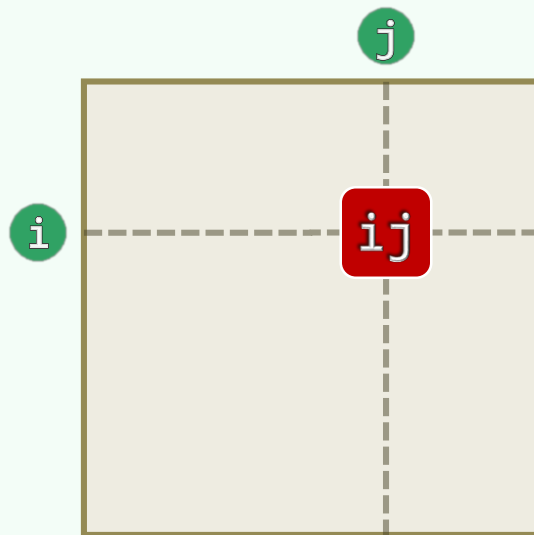
空间复杂度为 $\Theta(n^2)$, 与图中实际的边数无关

❖ incidence matrix: 记录**顶点**与**边**之间的**关联**关系

空间复杂度为 $\Theta(n * e) = O(n^3)$

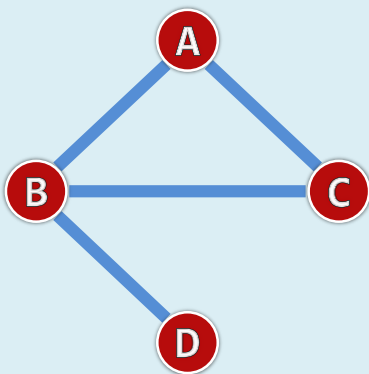
空间利用率 = $2e / ne = 2/n$

解决某些问题时十分有效



实例

(a) undigraph

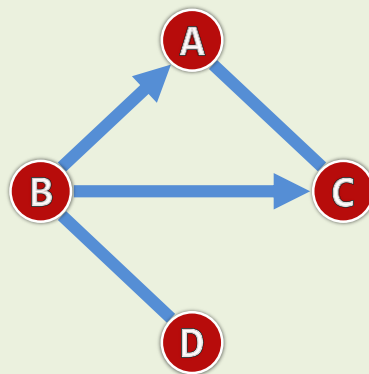


redundancy

对称矩阵

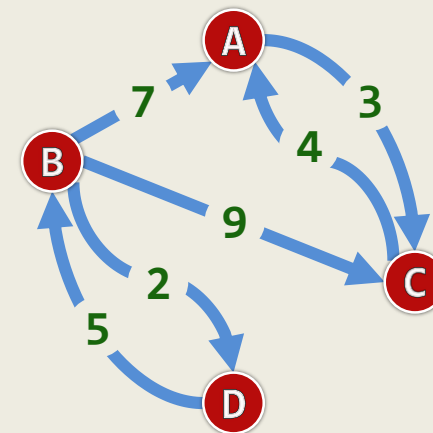
0	A	B	C	D
A		1	1	
B	1		1	1
C	1	1		
D		1		

(b) digraph



0	A	B	C	D
A			1	
B	1		1	1
C	1			
D		1		

(c) network



∞	A	B	C	D
A			3	
B	7		9	2
C	4			
D		5		

图

邻接矩阵：模版实现

Vertex

```
using VStatus = enum { UNDISCOVERED, DISCOVERED, VISITED };
```

```
template <typename Tv> struct Vertex { //不再严格封装
```

```
    Tv data; int inDegree, outDegree;
```

```
    VStatus status; // (如上三种) 状态
```

```
    int dTime, fTime; //时间标签 | 时间戳
```

dfs首次访问 discovery time
dfs结束访问 finish time

```
    Rank parent; //在遍历树中的父节点
```

```
    int priority; //在遍历树中的优先级 (最短通路、极短跨边等)
```

```
    Vertex( Tv const & d ) : //构造新顶点
```

```
        data( d ), inDegree( 0 ), outDegree( 0 ), status( UNDISCOVERED ),
```

```
        dTime( -1 ), fTime( -1 ), parent( -1 ), priority( INT_MAX ) {}
```

```
};
```

Edge

```
using EType = enum { UNDETERMINED, TREE, CROSS, FORWARD, BACKWARD };

template <typename Te> struct Edge { //不再严格封装

    Te data; //数据

    int weight; //权重 有权图

    EType type; //在遍历树中所属的类型

    Edge( Te const & d, int w ) : //构造新边

        data(d), weight(w), type(UNDETERMINED) {}

};
```

GraphMatrix

```
template <typename Tv, typename Te> class GraphMatrix : public Graph<Tv, Te> {  
private:
```

```
    Vector< Vertex<Tv> > V; //顶点集
```

= 维数组有边

```
    Vector< Vector< Edge<Te>* > > E; //边集
```

```
public: // 操作接口: 顶点相关、边相关、...
```

```
    GraphMatrix() { n = e = 0; } //构造
```

```
    ~GraphMatrix() { //析构
```

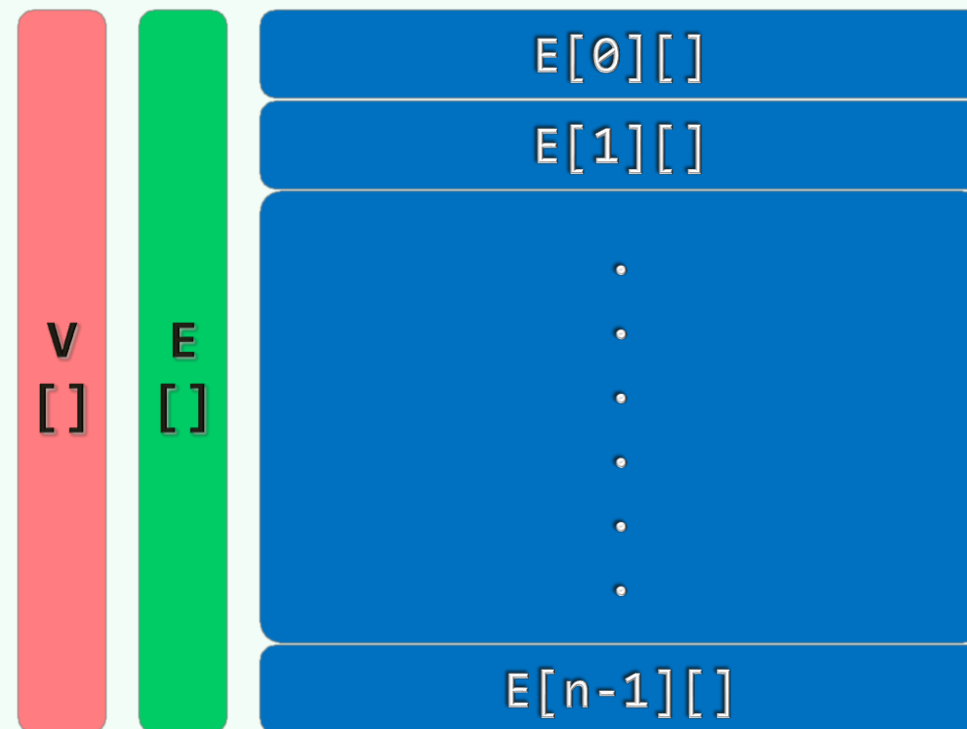
```
        for ( Rank j = 0; j < n; j++ )
```

```
            for ( Rank k = 0; k < n; k++ )
```

```
                delete E[j][k]; //清除所有边记录
```

```
    }
```

```
};
```



图

邻接矩阵：静态操作

顶点的读写

V

E

i

Tv & vertex(Rank i) { return V[i].data; } //数据

int inDegree(Rank i) { return V[i].inDegree; } //入度

int outDegree(Rank i) { return V[i].outDegree; } //出度

VStatus & status(Rank i) { return V[i].status; } //状态

int & dTime(Rank i) { return V[i].dTime; } //时间标签dTime

int & fTime(Rank i) { return V[i].fTime; } //时间标签fTime

Rank & parent(Rank i) { return V[i].parent; } //在遍历树中的父亲

int & priority(Rank i) { return V[i].priority; } //优先级数

边的读写

V

E

i

0

1

2

.....

j

.....

n-1

n

```
bool exists( Rank i, Rank j ) { //判断边(i, j)是否存在 (短路求值)
```

```
    return (i < n) && (j < n) && E[i][j] != NULL;
```

```
} //以下假定exists(i, j) = true
```

```
Te & edge( Rank i, Rank j ) { return E[i][j]->data; } //数值
```

```
EType & type( Rank i, Rank j ) { return E[i][j]->type; } //类型
```

```
int & weight( Rank i, Rank j ) { return E[i][j]->weight; } //权重
```

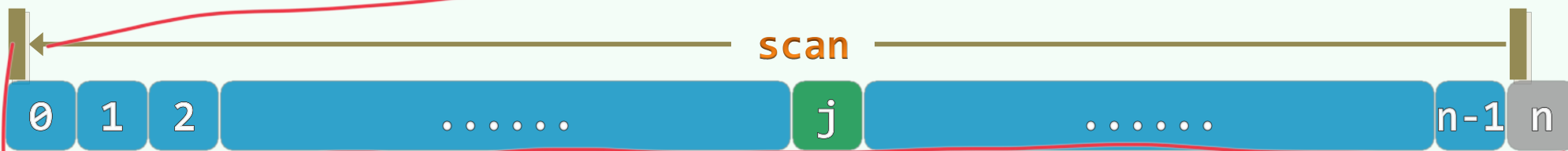
邻点的枚举

V

E

对于任意顶点*i*，如何枚举其所有的邻接顶点 (*neighbor*) ?

Rank firstNbr(Rank *i*) { return nextNbr(*i*, *n*); } //假想哨兵



Rank nextNbr(Rank *i*, Rank *j*) { //若已枚举至邻居*j*，则转向下一邻居

while (-1 < *j*) && ! exists(*i*, --*j*); //逆向顺序查找

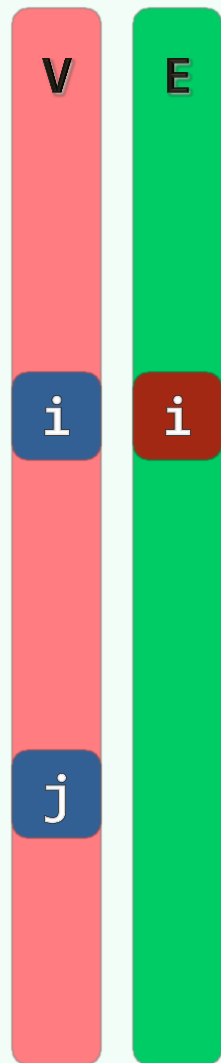
return *j*;

} //O(*n*)——改用邻接表，可提高至O(1 + outDegree(*i*))

图

邻接矩阵： 动态操作

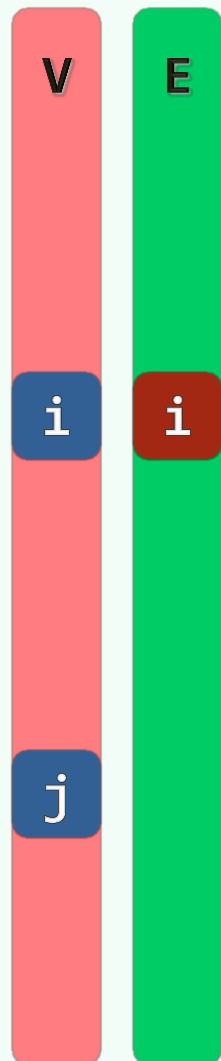
边的插入



```
void insert( Te const & edge, int w, Rank i, Rank j ) {  
    if ( exists(i, j) ) return; //忽略已有的边  
  
    E[i][j] = new Edge<Te>( edge, w ); //创建新边 (权重为w)  
  
    e++; //更新边计数  
  
    V[i].outDegree++; //更新顶点i的出度  
  
    V[j].inDegree++; //更新顶点j的入度  
}
```



边的删除



```
Te remove( Rank i, Rank j ) { //删除 (已确认存在的) 边(i, j)
```

```
    Te eBak = edge(i, j); //备份边(i, j)的信息
```

```
    delete E[i][j]; E[i][j] = NULL; //删除边(i, j)
```



```
    e--; //更新边计数
```

```
    V[i].outDegree--; //更新顶点i的出度
```

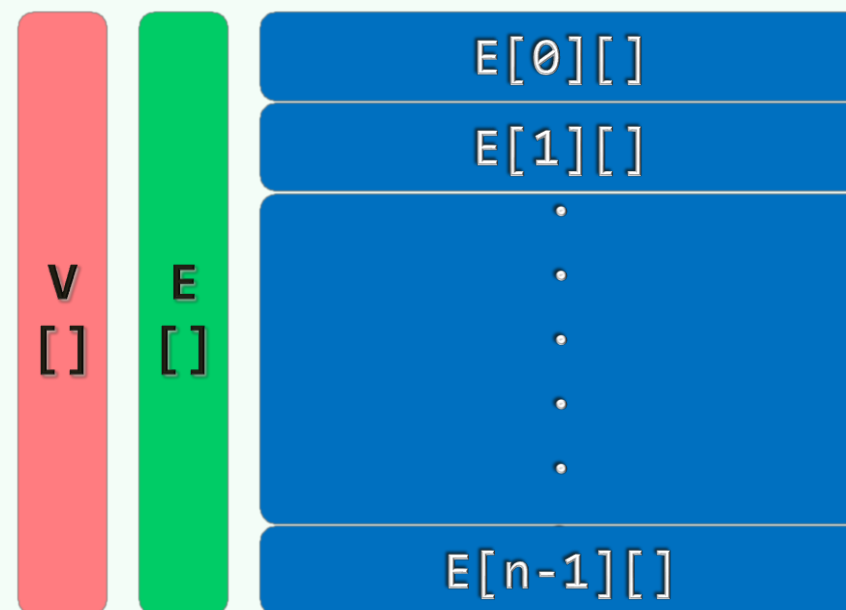
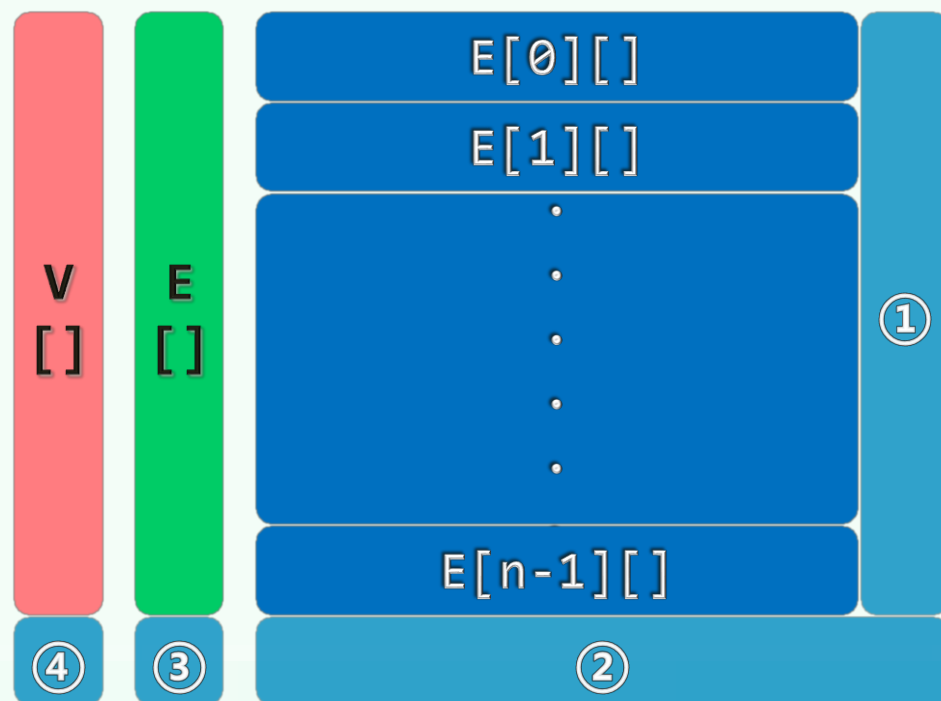
```
    V[j].inDegree--; //更新顶点j的入度
```

```
    return eBak; //返回被删除边的信息
```

```
}
```

顶点插入

```
Rank insert( Tv const & vertex ) { //插入顶点, 返回编号  
    for ( Rank j = 0; j < n; j++ ) E[j].insert( NULL ); n++; //①  
    E.insert( Vector< Edge<Te>* >( n, n, NULL ) ); //②③  
    return V.insert( Vertex<Tv>( vertex ) ); //④  
}
```



顶点删除

```
Tv remove( Rank i ) { //删除顶点及其关联边, 返回该顶点信息
    for ( Rank j = 0; j < n; j++ ) //删除所有出边
        if ( exists( i, j ) ) { delete E[i][j]; V[j].inDegree--; e-- }
    E.remove(i); n--; //删除第i行
    Tv vBak = vertex( i ); V.remove( i ); //备份之后, 删除顶点i
    for ( Rank j = 0; j < n; j++ ) //删除所有入边及第i列
        if ( Edge<Te> * x = E[j].remove( i ) )
            { delete x; V[j].outDegree--; e--; }
    return vBak; //返回被删除顶点的信息
}
```

图

邻接矩阵：性能分析

优点

❖ 直观，易于理解和实现

❖ 适用范围广泛

尤其适用于稠密图 (dense graph)

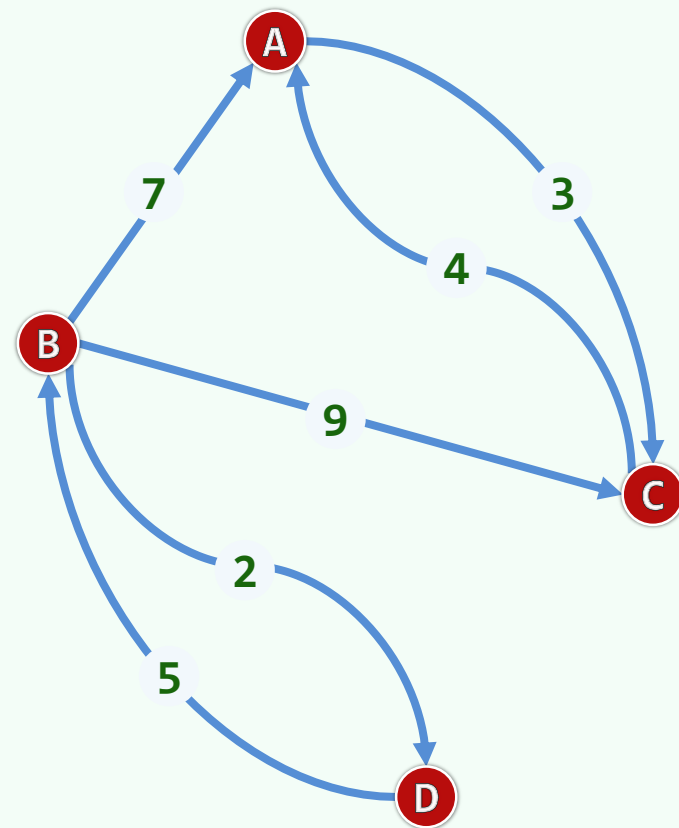
❖ 判断两点之间是否存在联边: $O(1)$

❖ 获取顶点的 (出/入) 度数: $O(1)$

添加、删除边后更新度数: $O(1)$

❖ 扩展性 (scalability) : 得益于Vector良好的控制策略, 空间溢出等情况可被“透明地”处理

∞	A	B	C	D
A			3	
B	7		9	2
C	4			
D		5		



缺点

- ❖ $\Theta(n^2)$ 空间，与边数无关！
- ❖ 真会有这么多条边吗？不妨考察一类特定的图...
- ❖ 平面图 (planar graph)：可**嵌入**于平面的图
- ❖ Euler's formula (1750):
$$v - e + f - c = 1, \text{ for any PG}$$
- ❖ 平面图： $e \leq 3 \times n - 6 = \mathcal{O}(n) \ll n^2$
此时，空间利用率 $\approx 1/n$
- ❖ 稀疏图 (sparse graph)
空间利用率同样很低，可采用压缩存储技术

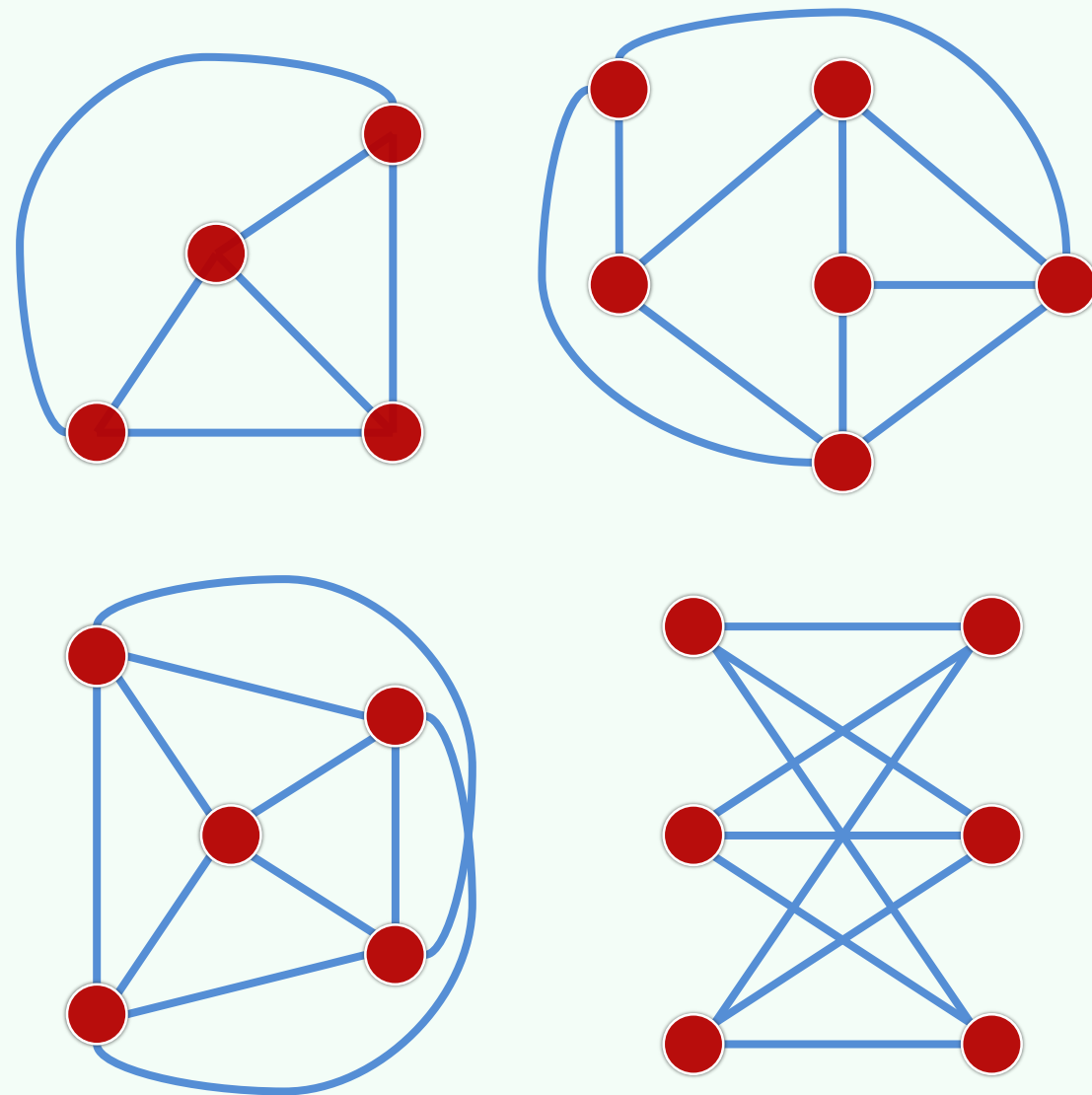
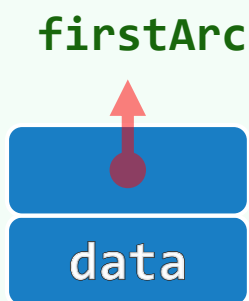


图 邻接表

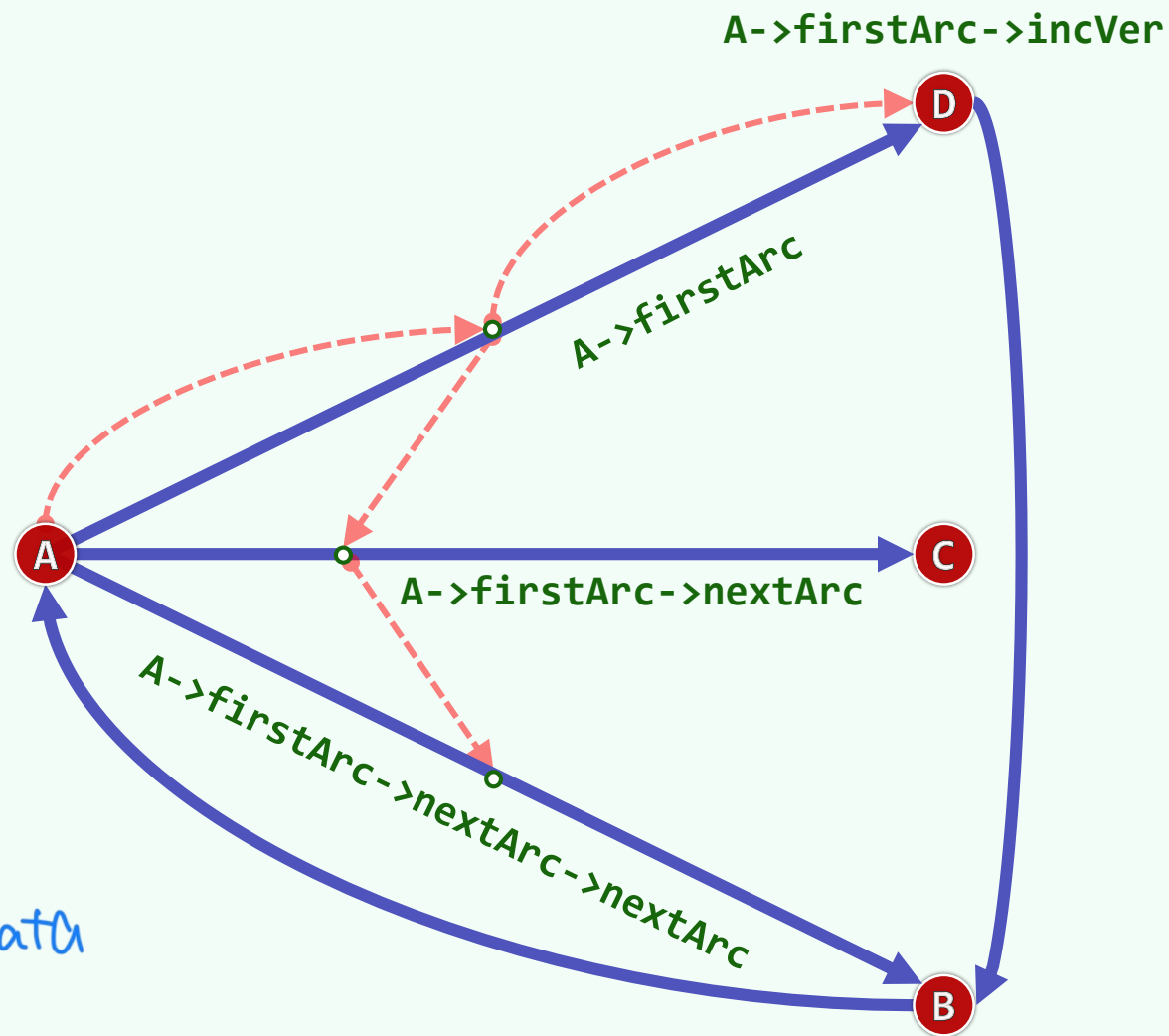
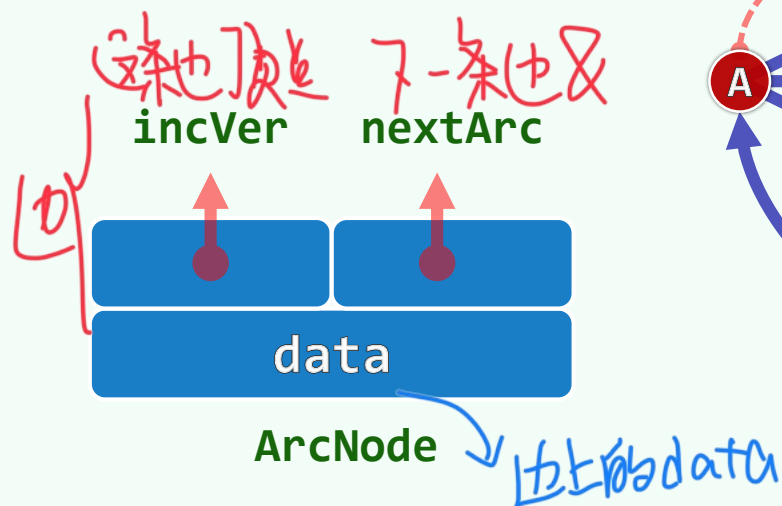
邻接表

- ❖ 如何避免邻接矩阵的空间浪费?
- ❖ 将邻接矩阵的各行组织为列表, 只记录存在的边
- ❖ 等效于, 每一顶点v对应于列表:

$$L_v = \{ u \mid \langle v, u \rangle \in E \}$$



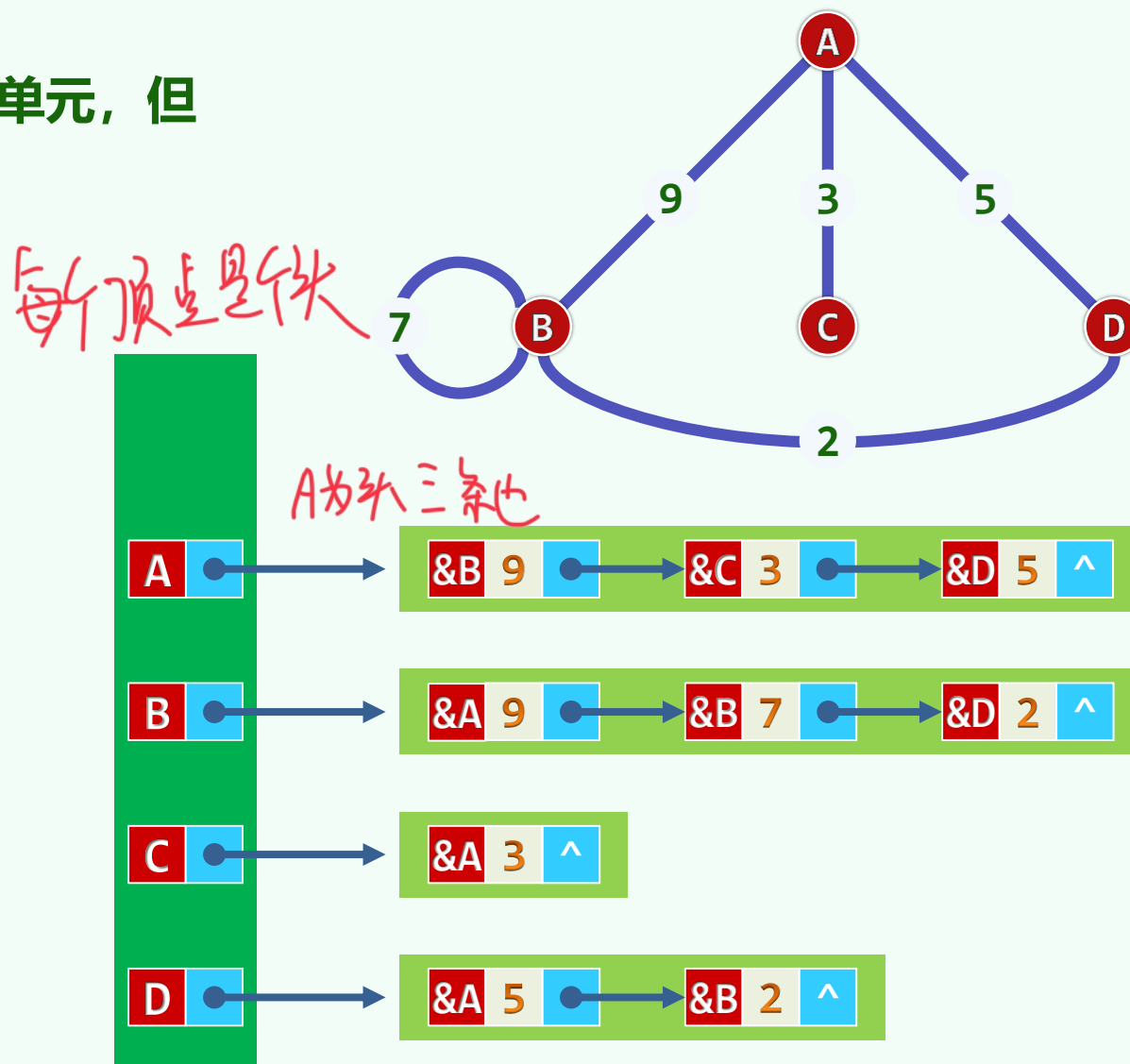
VNode
开表[链表]
ALGraph



实例

❖ 4个顶点，5条弧：不必占用 $4 \times 4 = 16$ 个单元，但
还是占用了9个单元，另加4个表头

∞	A	B	C	D
A		9	3	5
B	9	7		2
C	3			
D	5	2		



取舍原则

❖ 空间/速度

❖ 顶点类型

- bit
- int
- float
- struct
- class
- ...

❖ 弧类型 (方向 / 权值)

❖ 图类型 (稀疏 / 稠密)

	邻接矩阵	邻接表
适用场合	经常检测边的存在 经常做边的插入/删除 图的规模固定 稠密图	经常计算顶点的度数 顶点数目不确定 经常做遍历 稀疏图

图

广度优先搜索： 算法

Breadth-First Search

❖ 始自顶点s的广度优先搜索

bfs

访问顶点s

依次访问s所有**尚未访问**的邻接顶点

依次访问它们**尚未访问**的邻接顶点

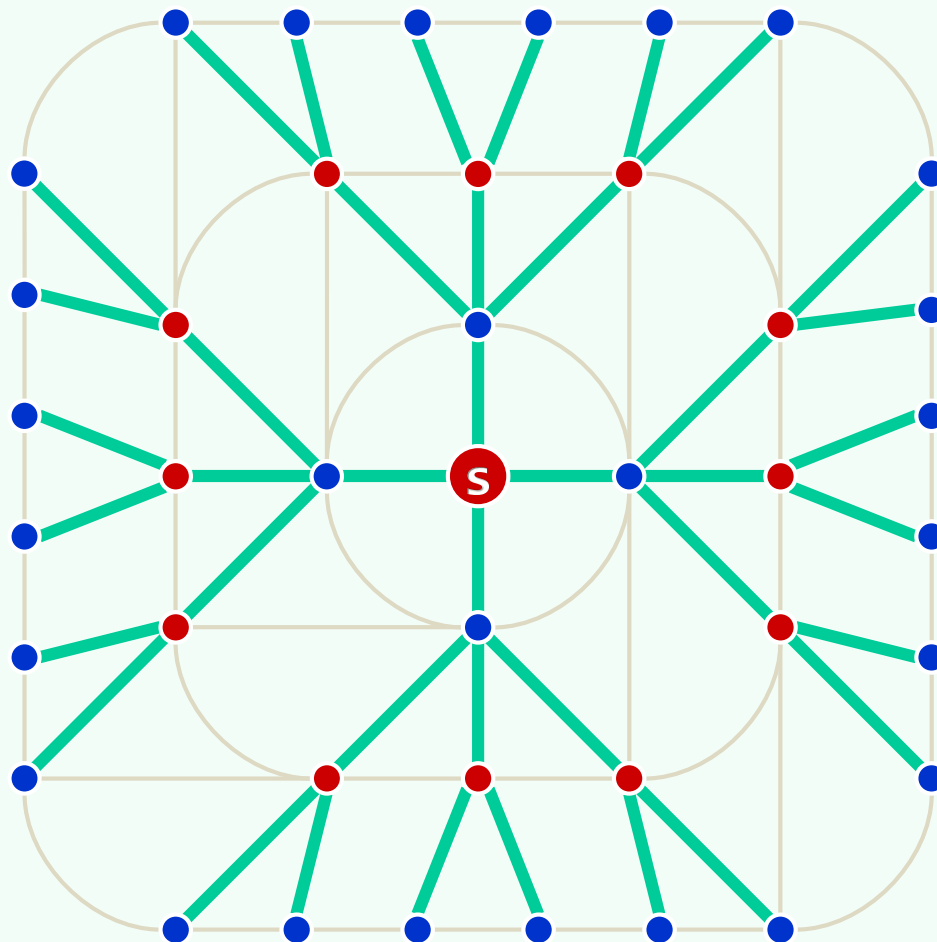
...

如此反复

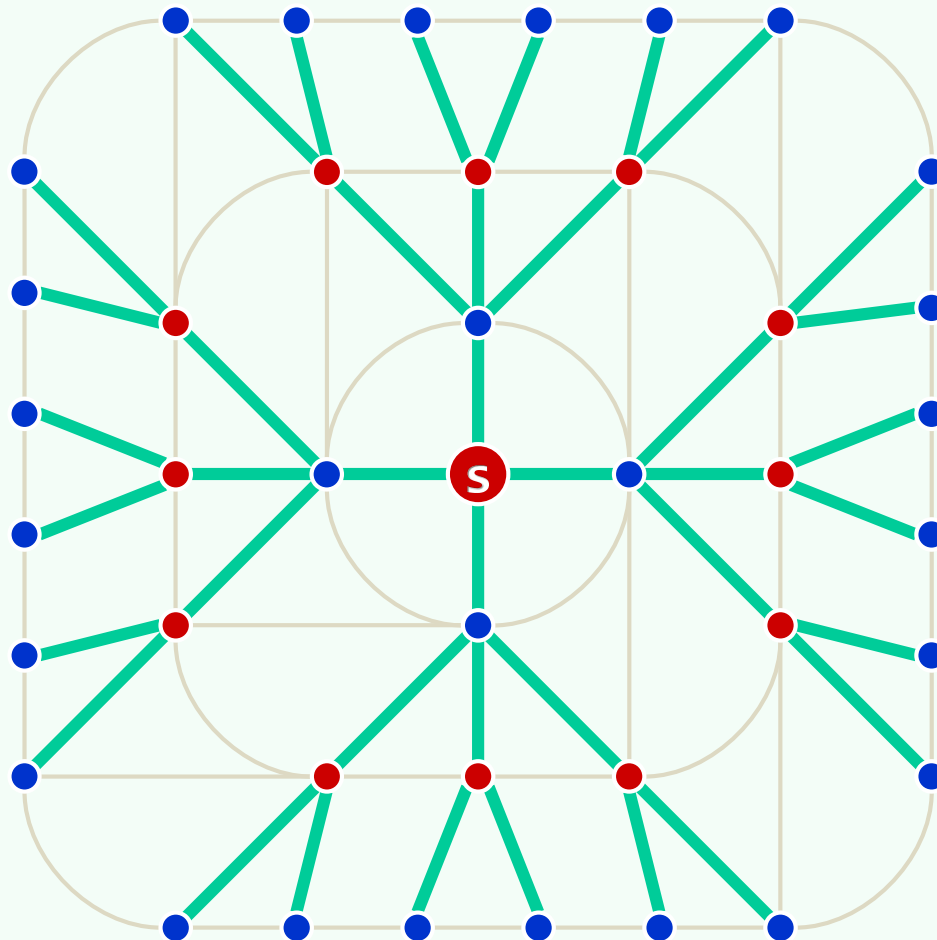
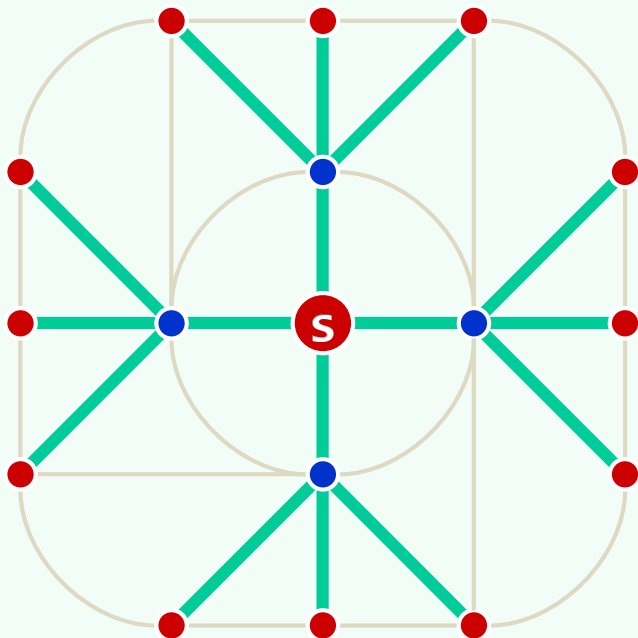
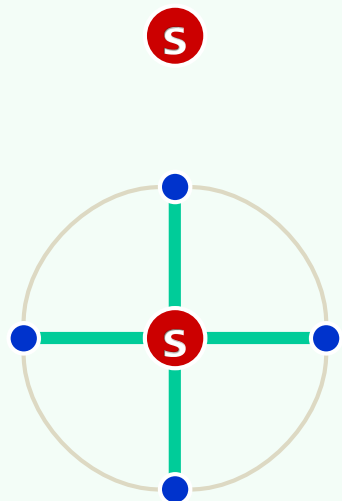
直至没有**尚未访问**的邻接顶点

❖ 以上策略及过程完全等同于树的**层次遍历**

❖ 事实上，BFS也的确会构造出原图的一棵支撑树（BFS tree）



譬喻：油气管线 + 绳网 + 涟漪



Graph::BFS() [1/2]

```
template <typename Tv, typename Te>
```

```
void Graph<Tv, Te>::BFS( Rank v, int & clock ) {
```

```
    Queue<Rank> Q; status(v) = DISCOVERED; Q.enqueue(v); //初始化
```

```
    while ( ! Q.empty() ) { //反复地
```

```
        ① Rank v = Q.dequeue(); dTime(v) = ++clock; //取出队首顶点v, 并
```

```
        for ( Rank u = firstNbr(v); -1 < u; u = nextNbr(v, u) ) //考查每一邻居u
```

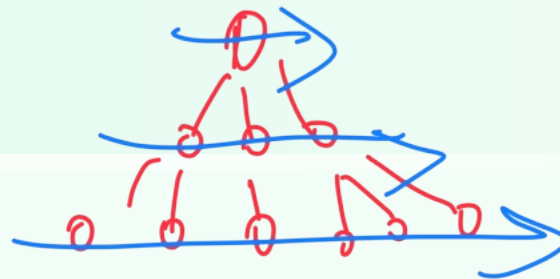
```
            /* ... 视u的状态, 分别处理 ... */
```

```
        ② status(v) = VISITED; //至此, 当前顶点访问完毕
```

```
    }
```

```
}
```

Graph::BFS() [2/2]



```
while ( ! Q.empty() ) { //反复地
```

```
    v Rank v = Q.dequeue(); dTime(v) = ++clock; //取出队首顶点v, 并
```

```
    for ( Rank u = firstNbr(v); -1 < u; u = nextNbr(v, u) ) //考查每一邻居u
```

```
        u if ( UNDISCOVERED == status(u) ) { //若u尚未被发现, 则
```

```
            u status(u) = DISCOVERED; Q.enqueue(u); //发现该顶点
```

```
                type(v, u) = TREE; parent(u) = v; //引入树边
```

```
        } else //若u已被发现 (正在队列中), 或者甚至已访问完毕 (已出队列), 则
```

```
            type(v, u) = CROSS; //将(v, u)归类于跨边
```

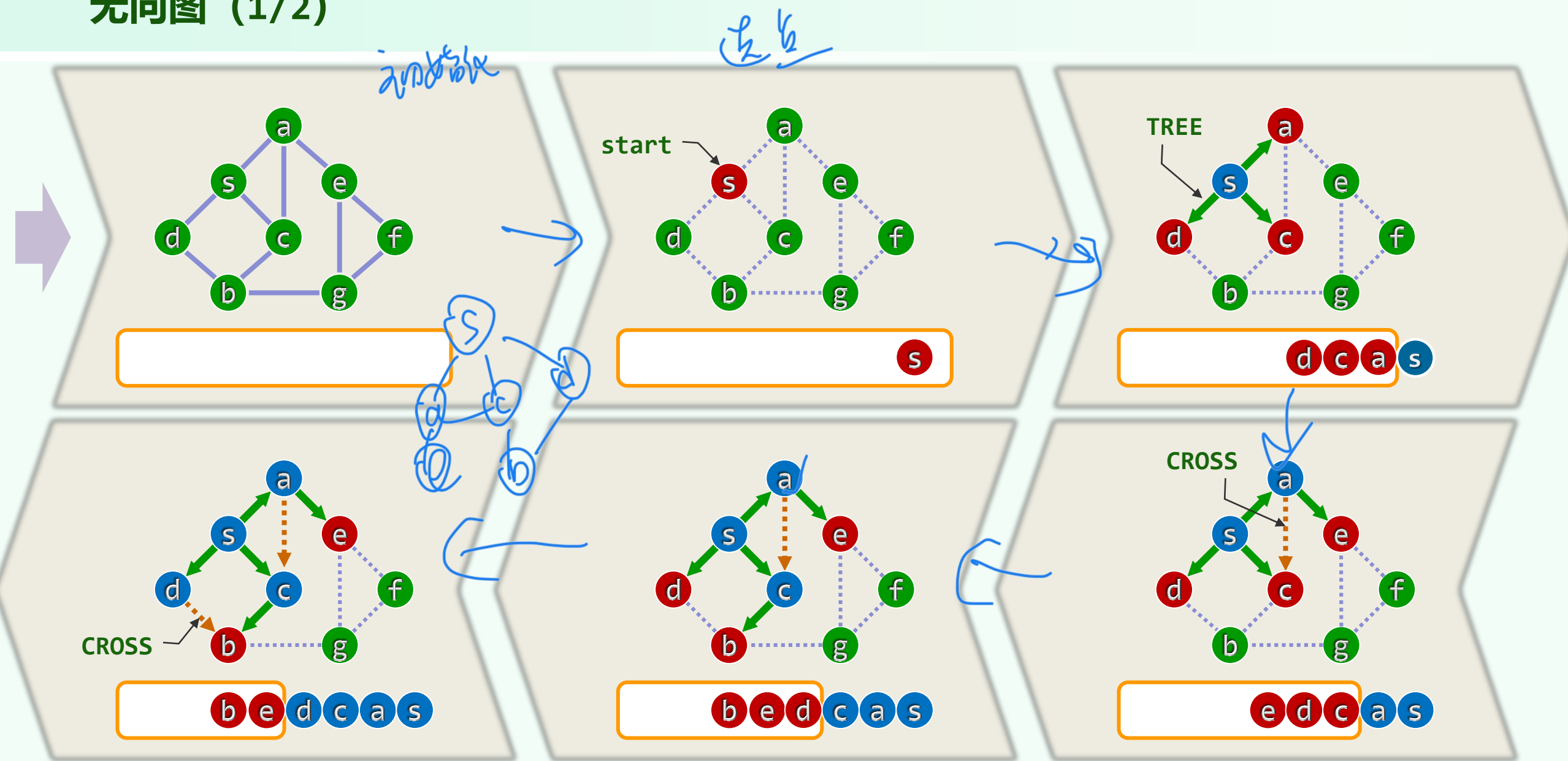
```
    v status(v) = VISITED; //至此, 当前顶点访问完毕
```

```
}
```

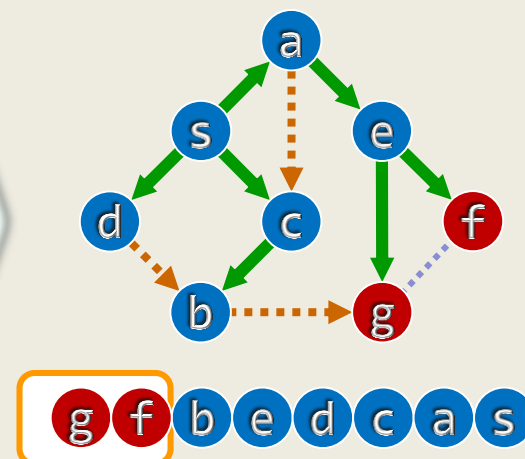
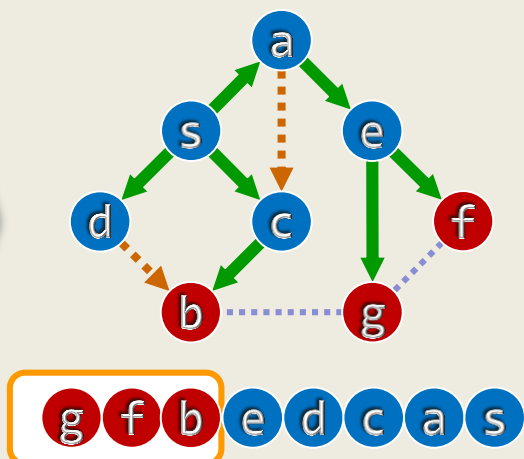
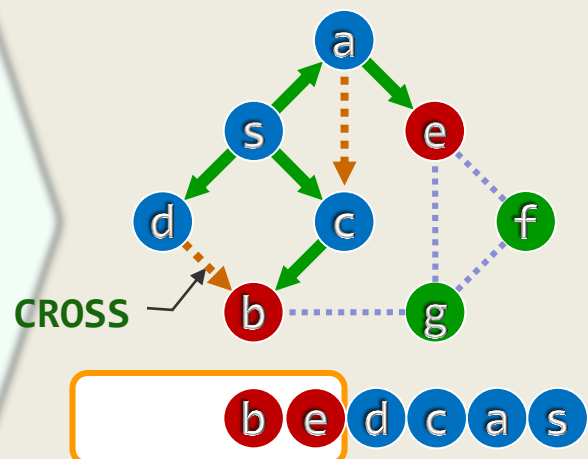
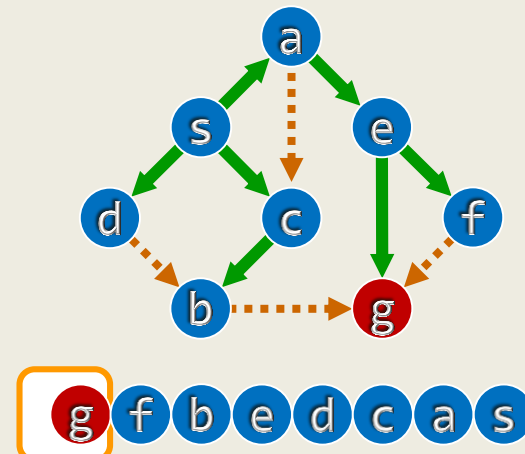
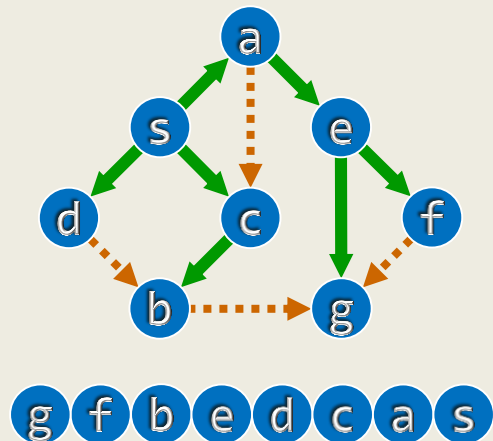
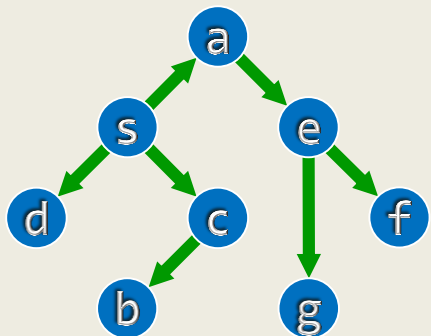
图

广度优先搜索：实例

无向图 (1/2)



无向图 (2/2)



图

广度优先搜索：推广

连通分量 + 可达分量

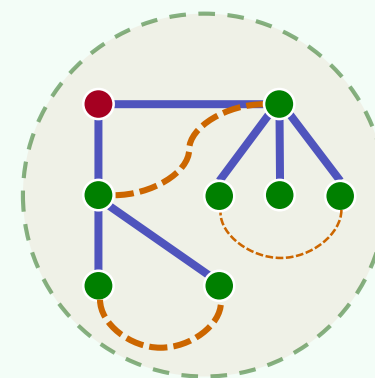
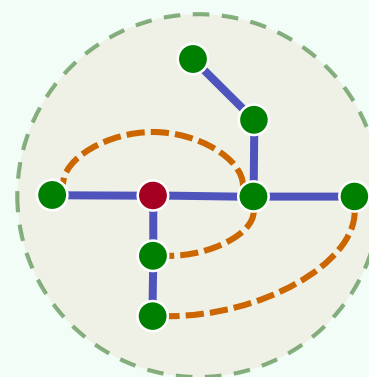
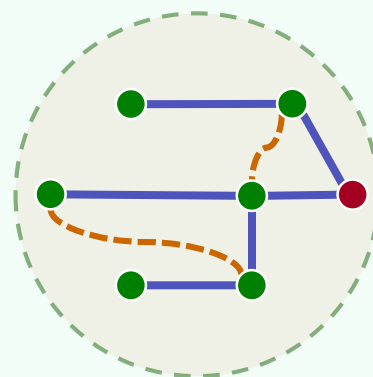
❖ 问题

- 给定**无向图**，找出其中任一顶点s所在的**连通图**
- 给定**有向图**，找出源自其中任一顶点s的**可达分量**

❖ 算法

- 从s出发做BFS
- 输出所有**被发现**的顶点
- 队列为空后立即终止，无需考虑其它顶点

❖ 若图中包含**多个**连通/可达分量，又该如何保证对**全图**的遍历呢？



Graph::bfs()

❖ template <typename Tv, typename Te>

```
void Graph<Tv, Te>::bfs( Rank s ) { //s为起始顶点
```

```
    reset(); int clock = 0; Rank v = s; //初始化 $\Theta(n+e)$ 
```

```
    do //逐一检查所有顶点，一旦遇到尚未发现的顶点
```

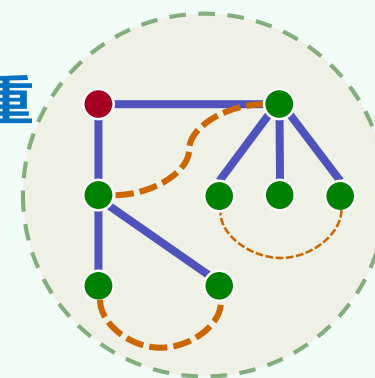
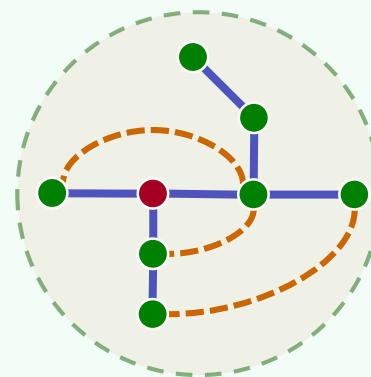
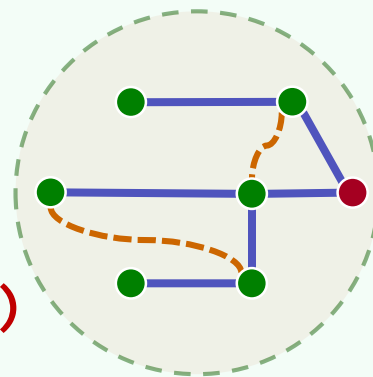
```
        if ( UNDISCOVERED == status(v) ) //累计 $\Theta(n)$ 
```

```
            BFS( v, clock ); //即从该顶点出发启动一次BFS
```

```
        while ( s != ( v = ( ( v+1 ) % n ) ) ); //按序号访问，不漏不重
```

```
    } //无论共有多少连通/可达分量...
```

❖ bfs()均可遍历它们，而且自身累计仅需线性时间...



复杂度

❖ 考查无向图...

$O(\text{顶点} + e)$

❖ bfs()的初始化 (reset()) : $O(n + e)$

❖ BFS()的迭代

- 外循环 (`while ( !Q.empty() )`)

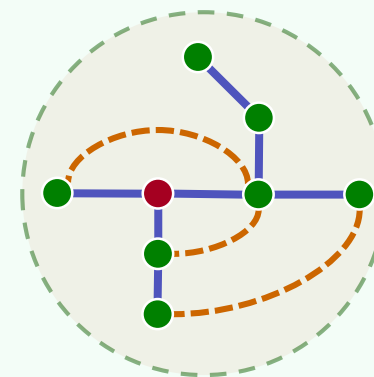
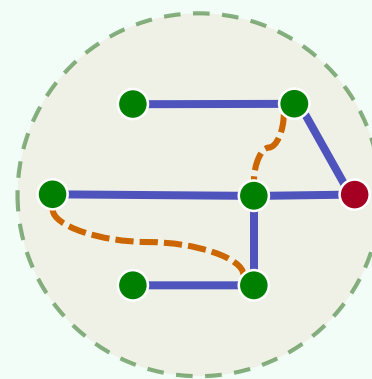
每个顶点各进入1次

- 内循环 (枚举v的每一邻居) : $O(1 + \text{deg}(v))$

- 总共: $O(\sum_{v \in V} (1 + \text{deg}(v))) = O(n + 2e)$

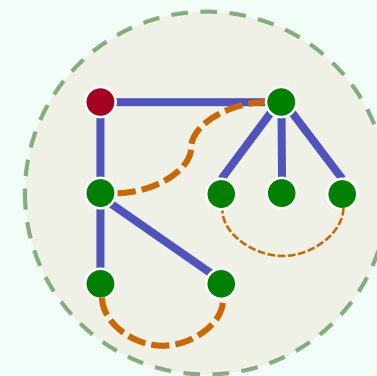
❖ 整个算法: $O(n + e) + O(n + 2e) = O(n + e)$

❖ 有向图呢? 亦是如此!



出度+自身
(改用邻接表) 邻接矩阵 $O(n^2)$

每个顶点都做
 $\downarrow$
 n 也做2次: $2e$



图

广度优先搜索：性质及应用

边分类

❖ 经BFS后，所有边将确定方向，且被分为两类

❖ (v, u) 被标记为 **TREE** 时， v 为 **DISCOVERED** 且 u 为 **UNDISCOVERED**



❖ (v, u) 被标记为 **CROSS** 时， v 和 u 均为 **DISCOVERED** 或者 v 为 **DISCOVERED** 而 u 为 **VISITED**



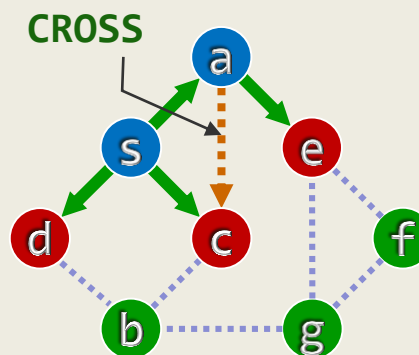
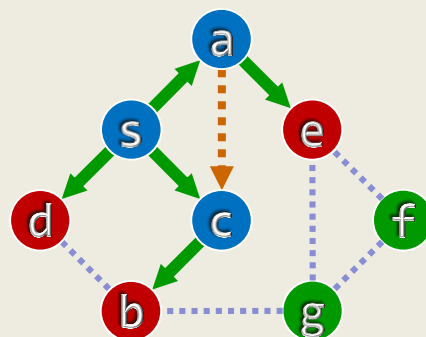
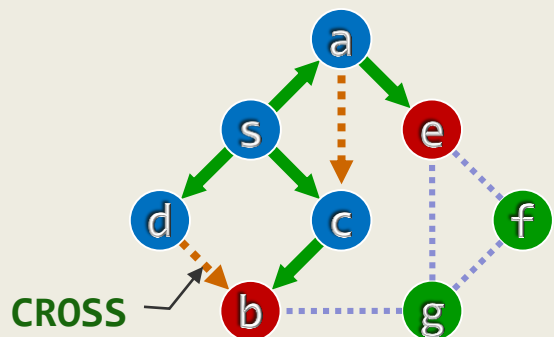
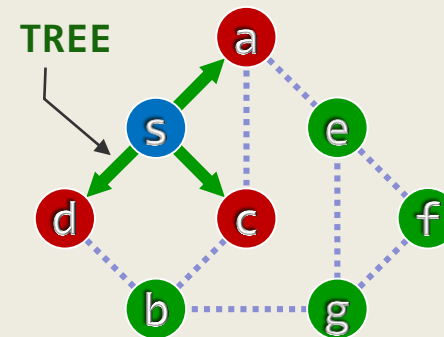
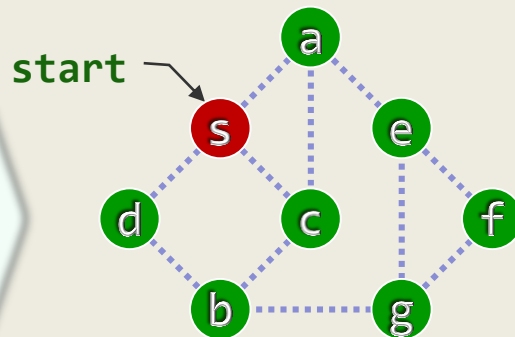
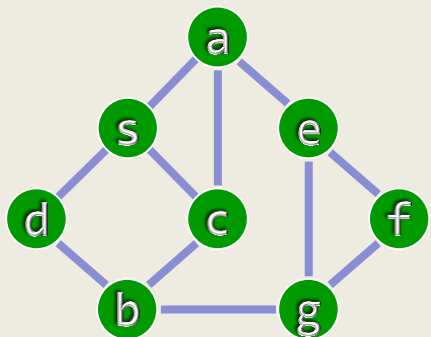
$a \rightarrow c / d \rightarrow b$

不同边下是什么情况呢？



不论 (v, u) 是有向边或无向边，两种情况均可能出现

无向图 (1/2)



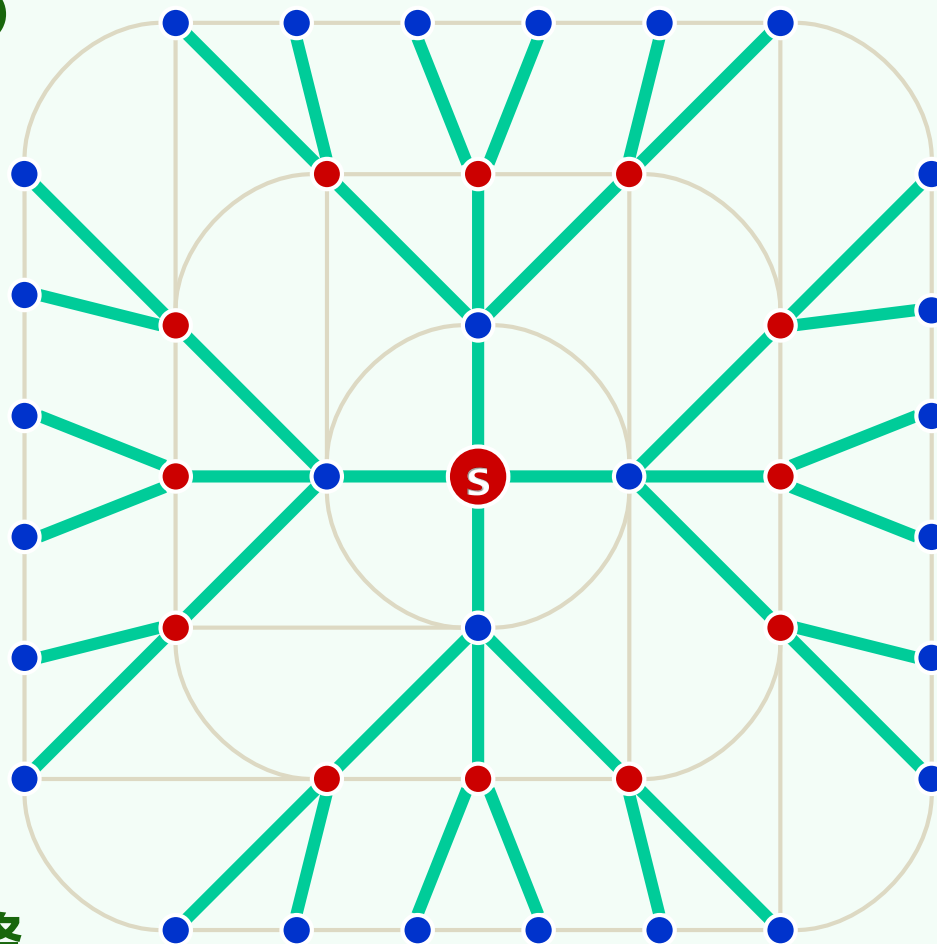
最短路径

❖ 无向图中，顶点 v 到 u 的（最近）距离记作 $\text{dist}(v, u)$

❖ BFS过程中，队列 Q 犹如一条贪吃蛇

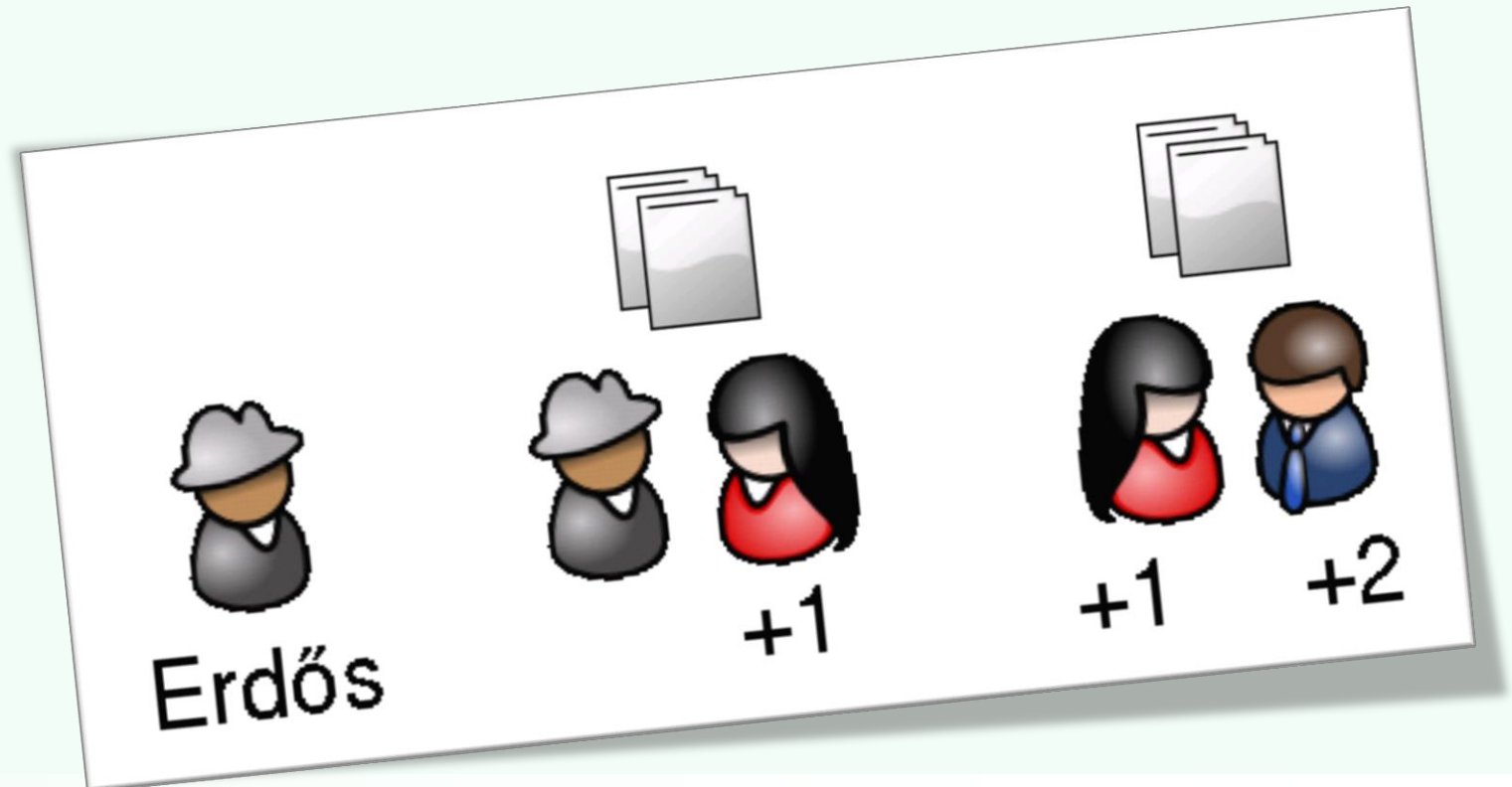
- 其中的顶点按 $\text{dist}(s)$ 单调排列
- 相邻顶点的 $\text{dist}(s)$ 相差不超过1
- 首、末顶点的 $\text{dist}(s)$ 相差不超过1
- 由树边联接的顶点， $\text{dist}(s)$ 恰好相差1
- 由跨边联接的顶点， $\text{dist}(s)$ 至多相差1

❖ BFS树中从 s 到 v 的路径，即是二者在原图中的最短通路



Erdős Number

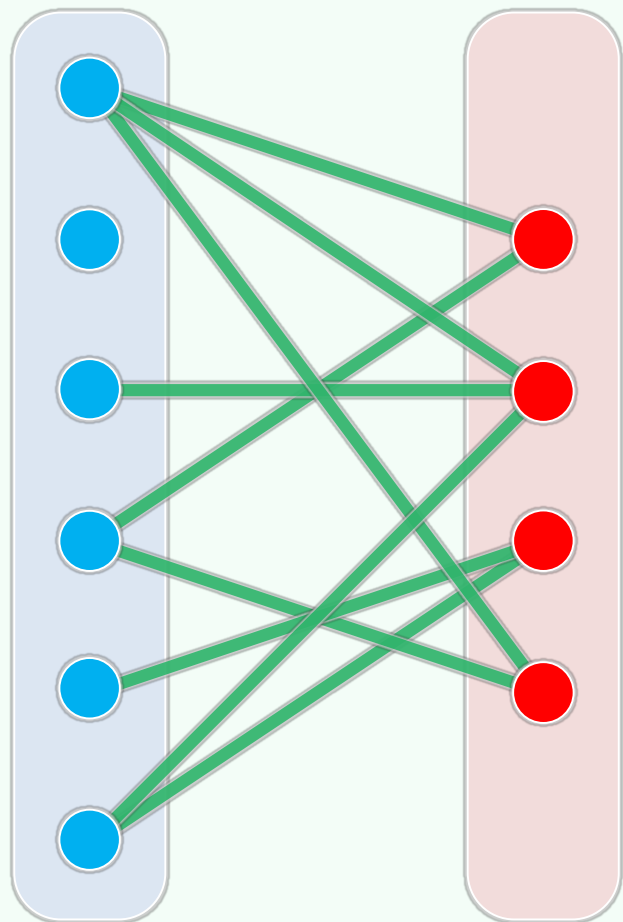
- ❖ Describes the "collaborative distance" between mathematician Paul Erdős and another person, as measured by authorship of mathematical papers



Bipartite Graph (Bigraph)

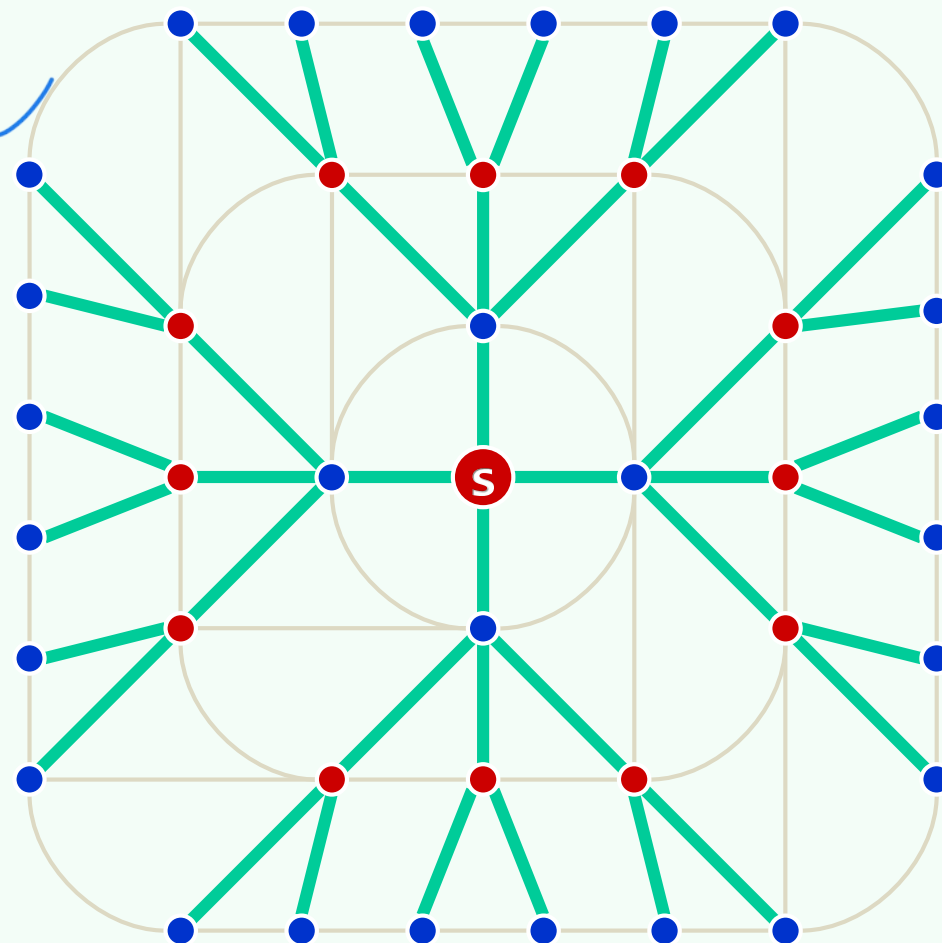
二部图

改进 bfs 判断是否合法二部图



发现新邻居
undiscovered 着色

else 比对
(邻居已被着色)



图

深度优先搜索： 算法

Depth-First Search

❖ DFS(s) // 始自顶点 s 的深度优先搜索

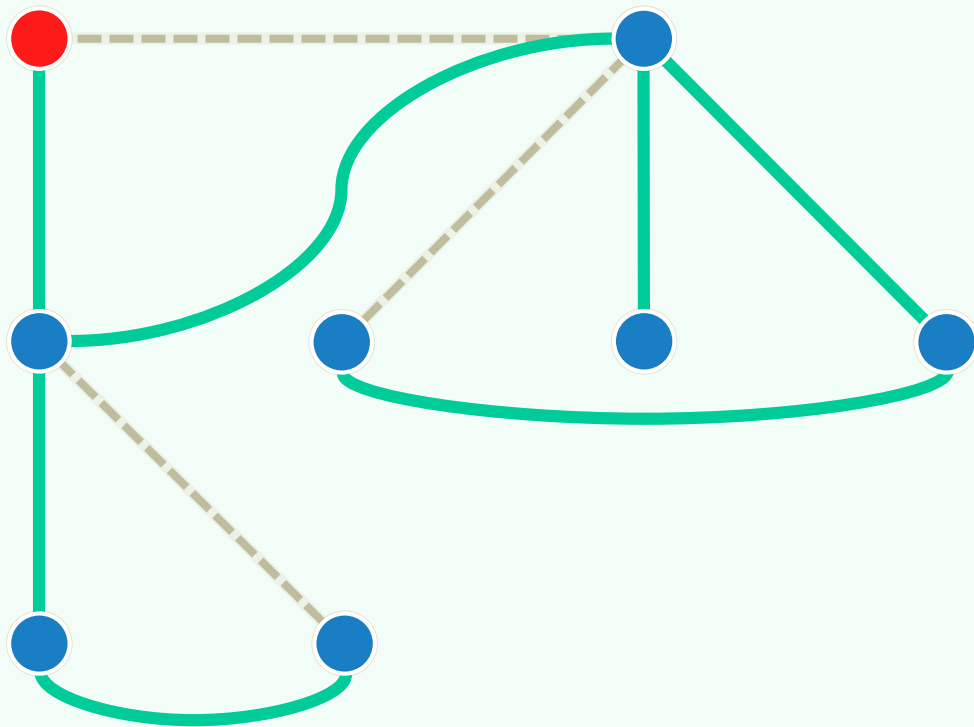
- 访问顶点 s
- 若 s 尚有未被访问的邻居, 则
任取其一 u , 递归执行DFS(u)
- 否则, 返回

❖ 若此时尚有顶点未被访问

任取这样的一个顶点作起始点

❖ 重复上述过程, 直至所有顶点都被访问到

❖ 对树而言, 等效于先序遍历: DFS也的确会构造出原图的一棵支撑树 (DFS tree)



Graph::DFS() [1/2]

```
template <typename Tv, typename Te>
```

```
void Graph<Tv, Te>::DFS( Rank v, int & clock ) {
```

```
    v dTime(v) = ++clock; status(v) = DISCOVERED; //发现当前顶点v
```

开始被访问时间
discovery time

```
    for ( Rank u = firstNbr(v); -1 < u; u = nextNbr(v, u) ) //考察v的每一邻居u
```

```
        /* ... 视u的状态, 分别处理 ... */
```

```
        /* ... 与BFS不同, 含有递归 ... */
```

```
    v status(v) = VISITED; fTime(v) = ++clock; //至此, 当前顶点v方告访问完毕
```

结束时间

```
}
```

Graph::DFS() [2/2]

```
for ( Rank u = firstNbr(v); -1 < u; u = nextNbr(v, u) ) //考查v的每一邻居u
```

```
switch ( status(u) ) { //并视其状态分别处理
```

① case **UNDISCOVERED**: //u尚未发现, 意味着支撑树可在此拓展

```
type(v, u) = TREE; parent(u) = v; DFS( u, clock ); break; //递归
```

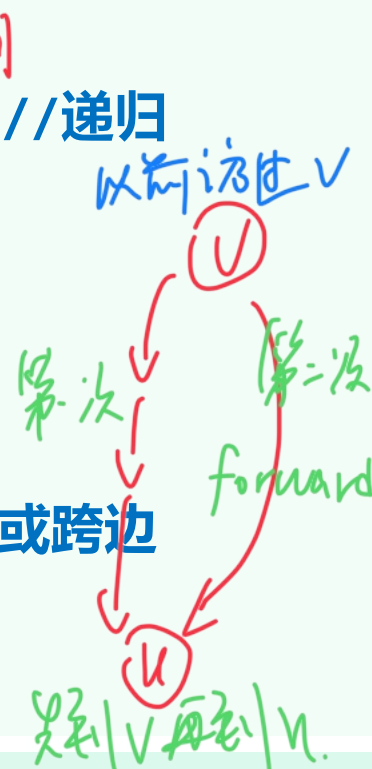
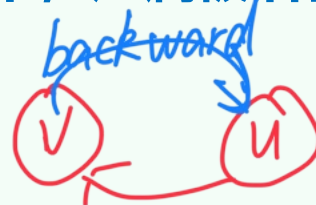
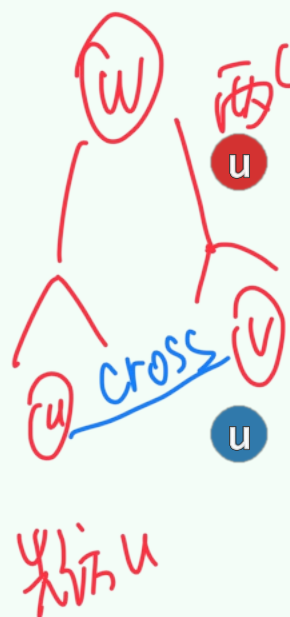
② case **DISCOVERED**: //u已被发现但尚未访问完毕, 应属被后代指向的祖先

```
type(v, u) = BACKWARD; break;
```

③ default: //u已访问完毕 (**VISITED**, 有向图), 则视承袭关系分为前向边或跨边

```
type(v, u) = dTime(v) < dTime(u) ? FORWARD : CROSS; break;
```

```
} //switch
```

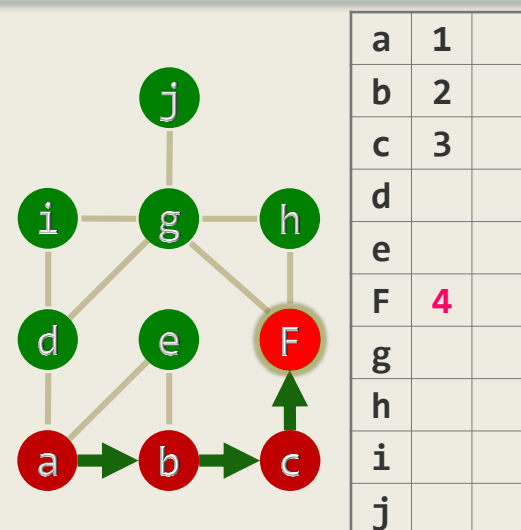
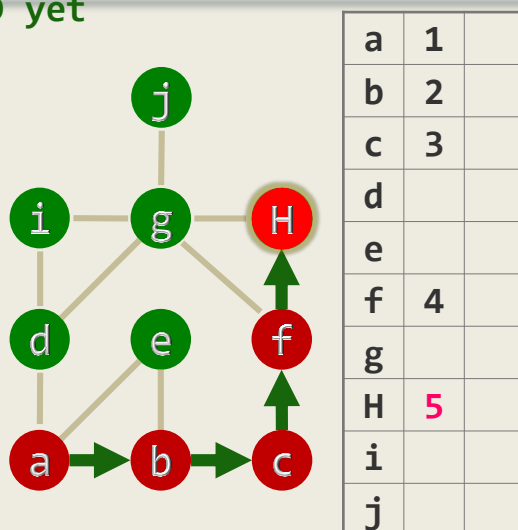
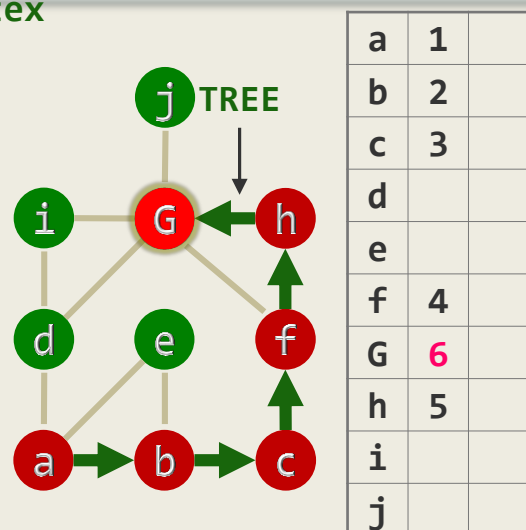
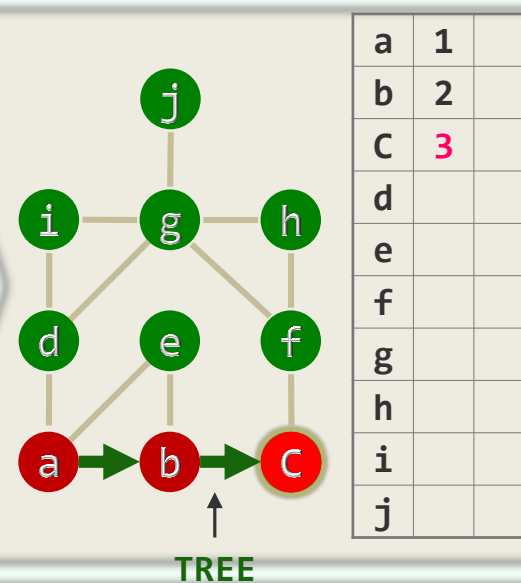
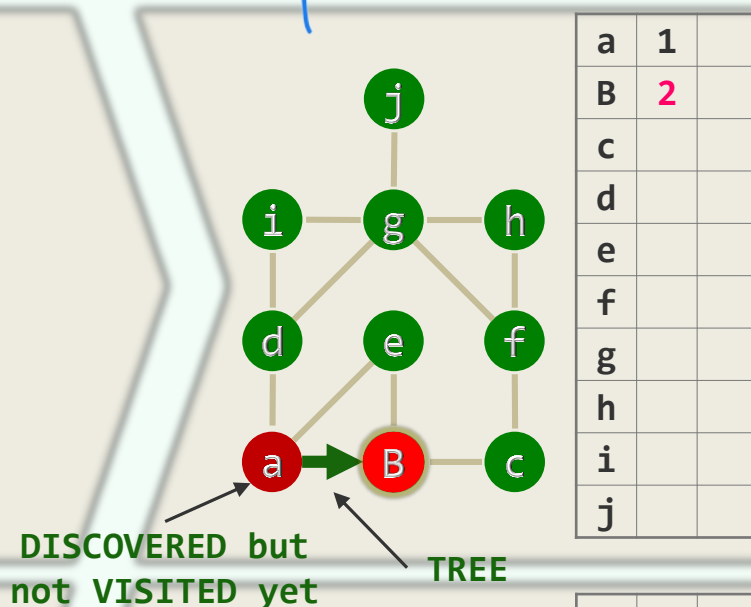
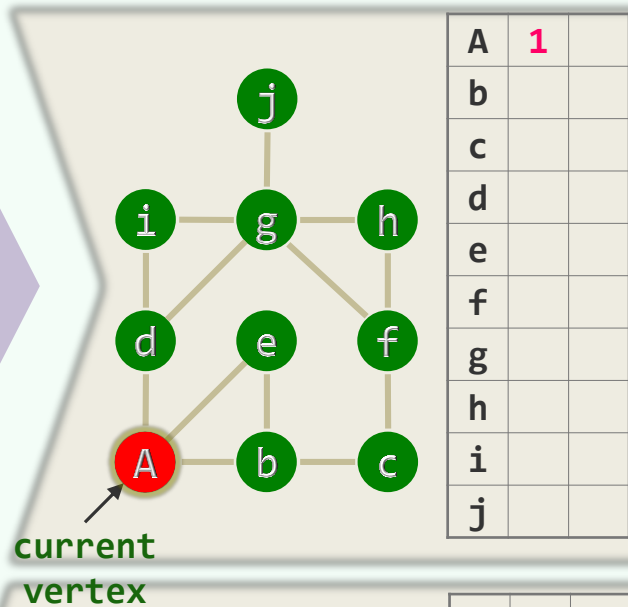


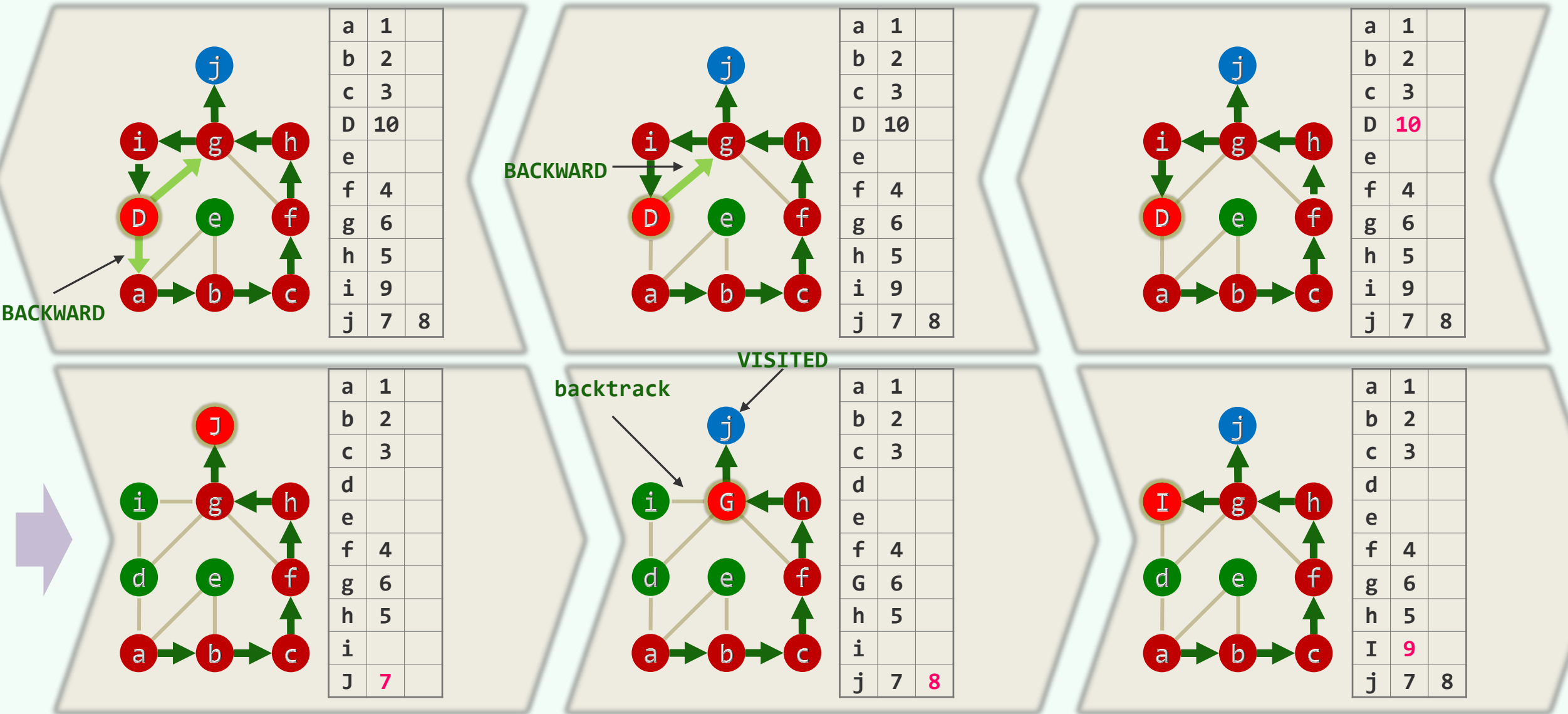
图

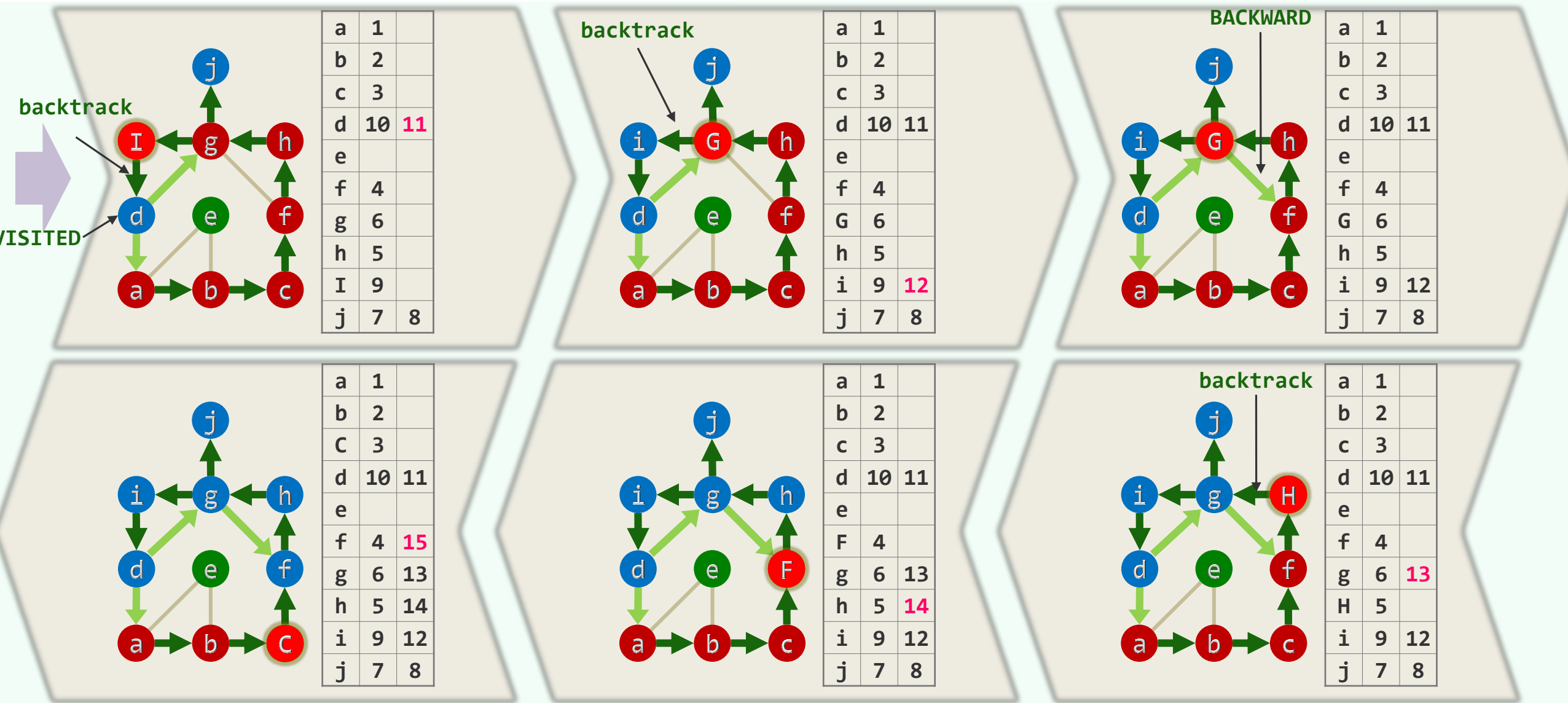
深度优先搜索：实例（无向图）

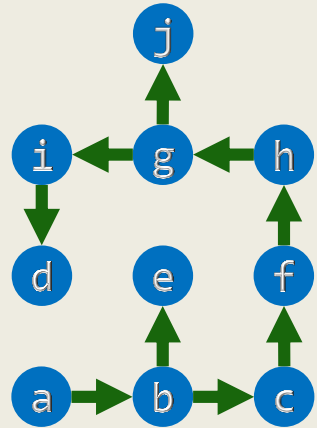
1/4

backward 也实现

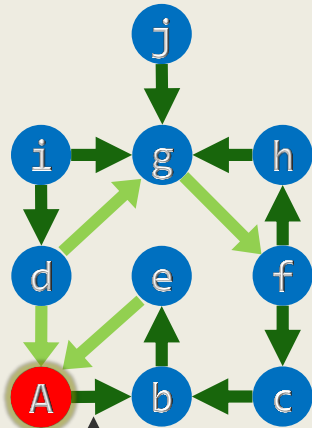






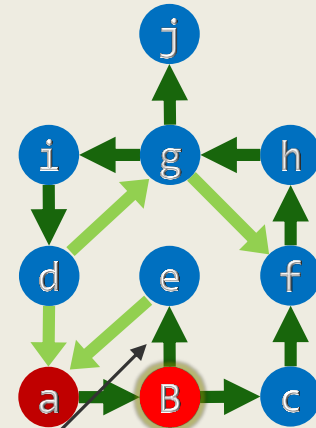


a	1	20
b	2	19
c	3	16
d	10	11
e	17	18
f	4	15
g	6	13
h	5	14
i	9	12
j	7	8



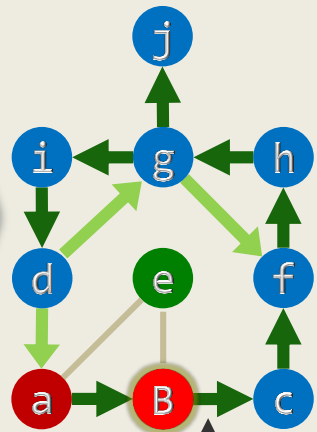
A	1	
b	2	19
c	3	16
d	10	11
e	17	18
f	4	15
g	6	13
h	5	14
i	9	12
j	7	8

backtrack



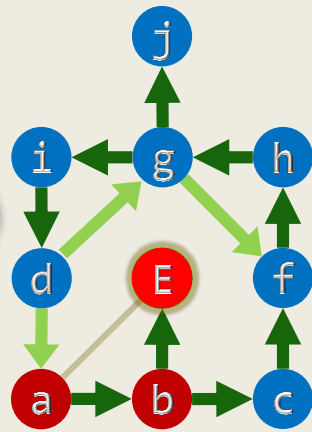
a	1	
B	2	
c	3	16
d	10	11
e	17	18
f	4	15
g	6	13
h	5	14
i	9	12
j	7	8

backtrack

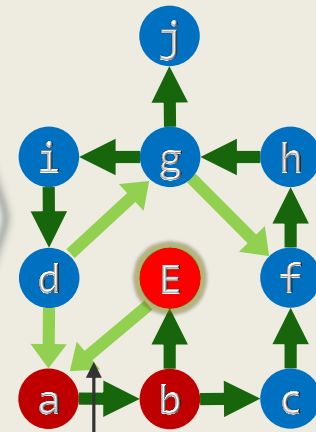


a	1	
B	2	
c	3	16
d	10	11
e		
f	4	15
g	6	13
h	5	14
i	9	12
j	7	8

backtrack



a	1	
b	2	
c	3	16
d	10	11
E	17	
f	4	15
g	6	13
h	5	14
i	9	12
j	7	8



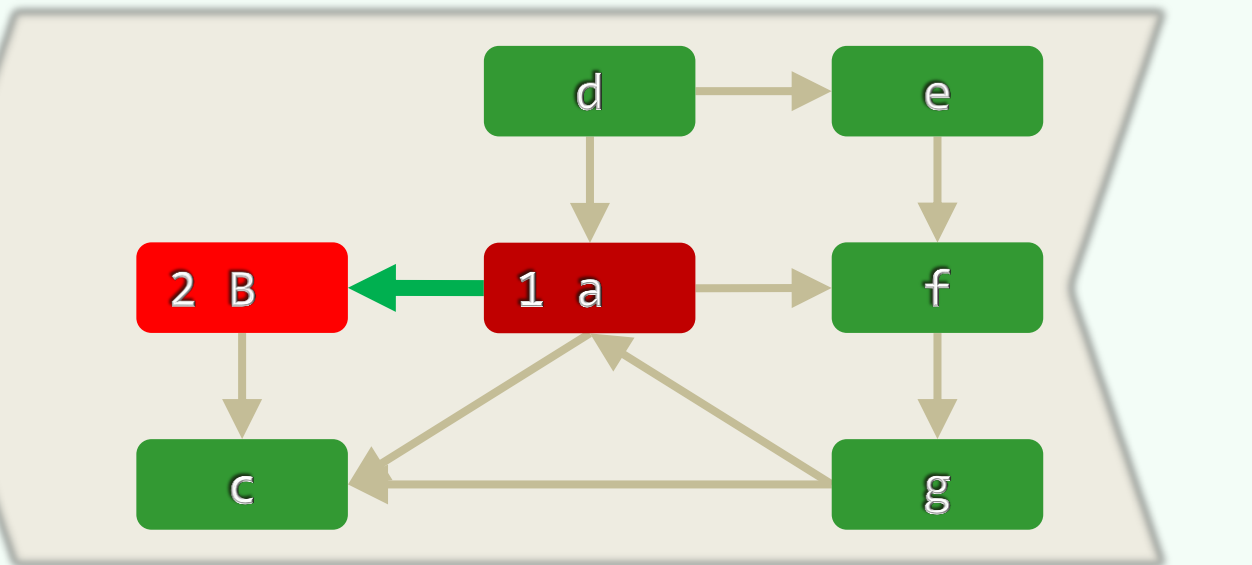
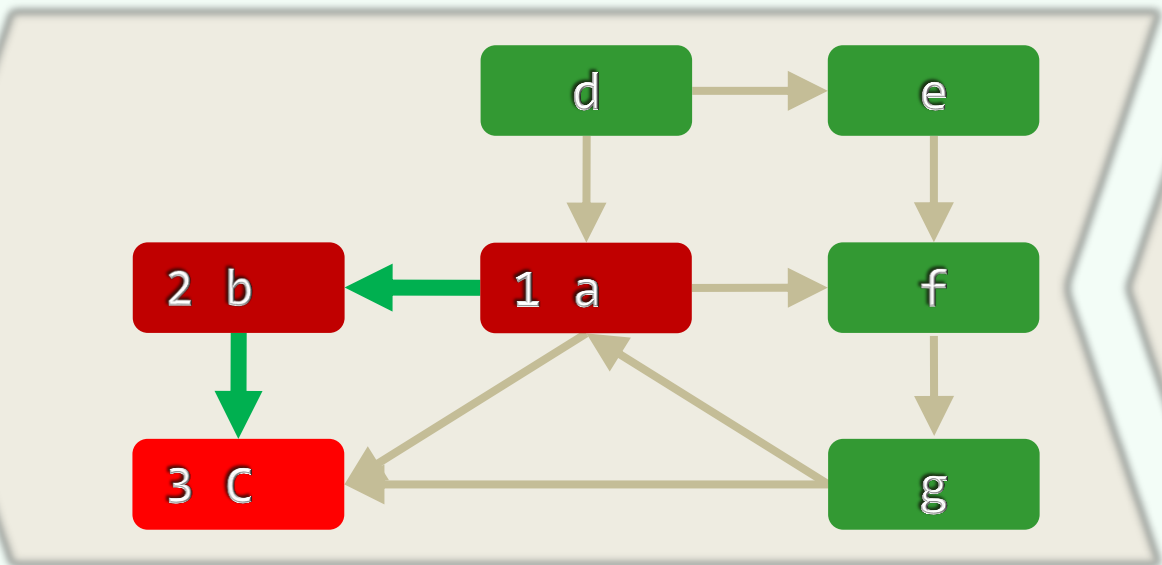
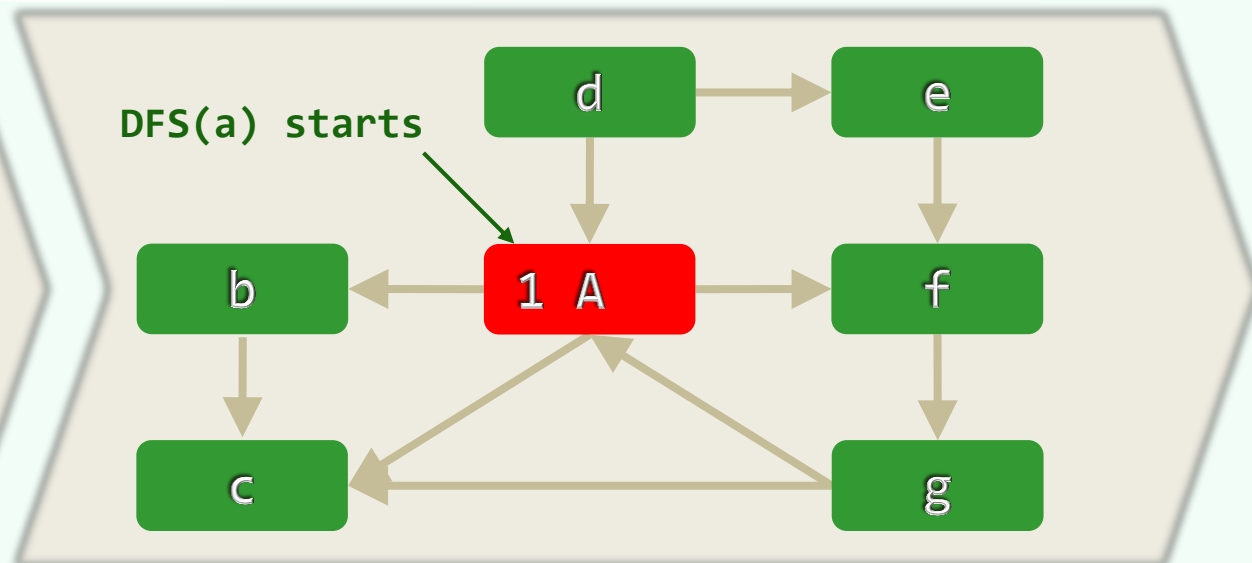
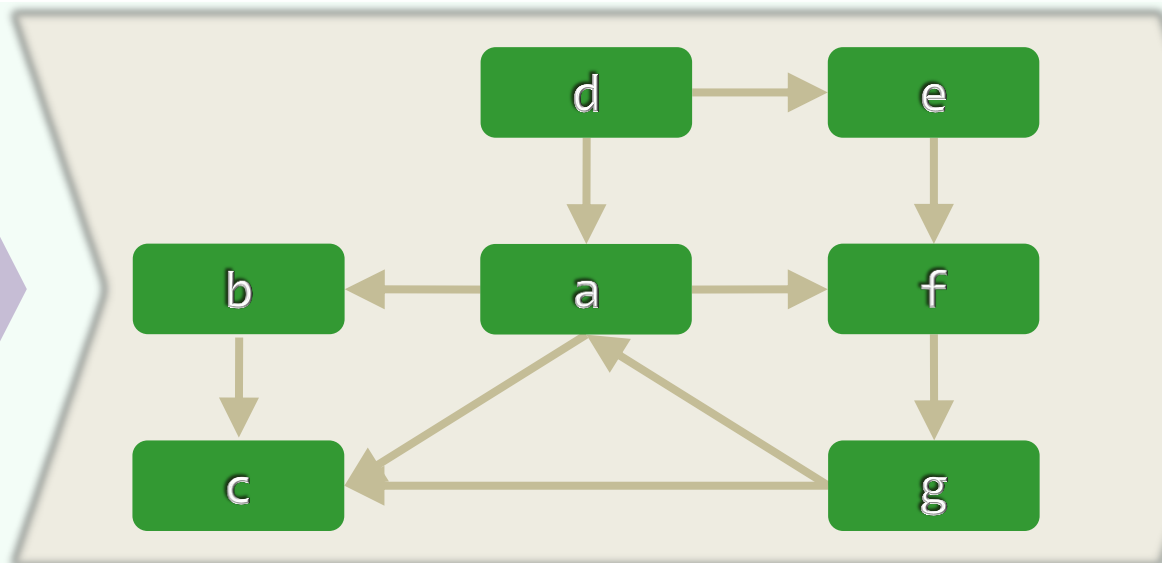
a	1	
b	2	
c	3	16
d	10	11
E	17	
f	4	15
g	6	13
h	5	14
i	9	12
j	7	8

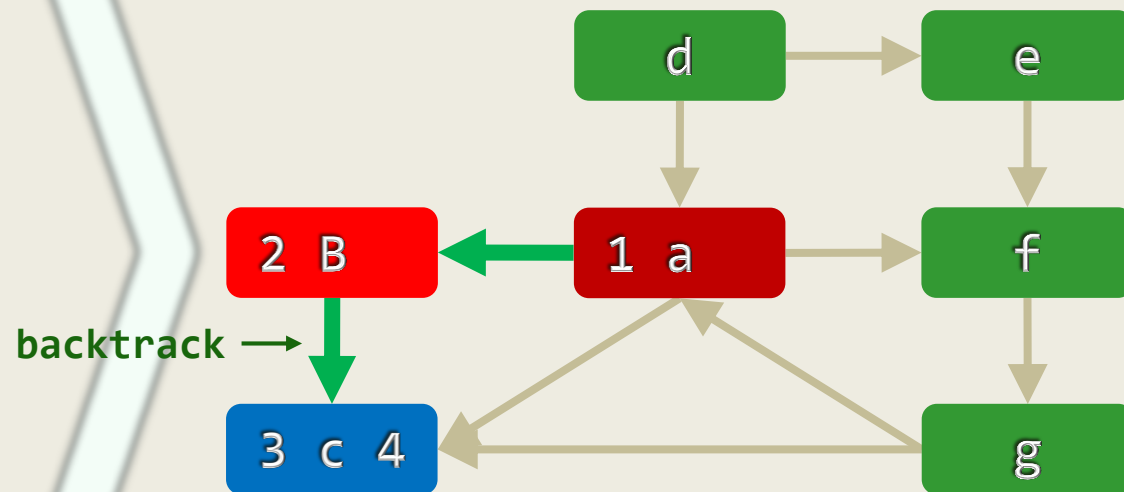
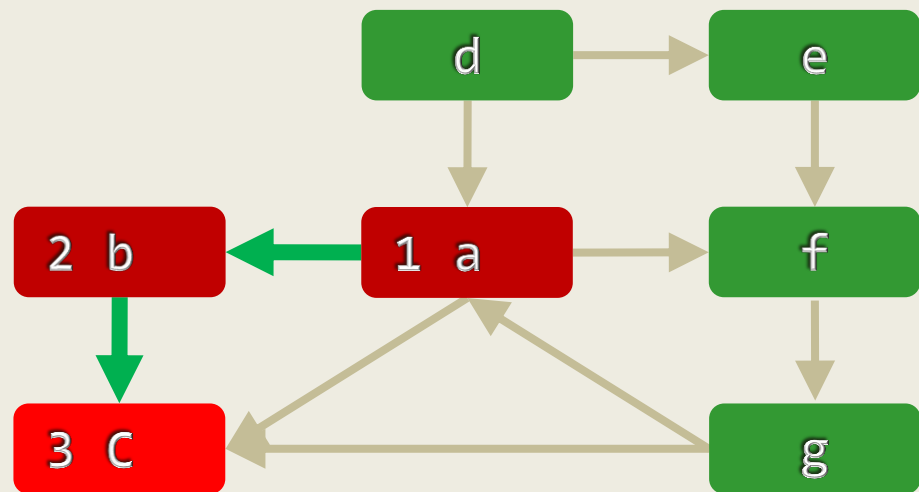
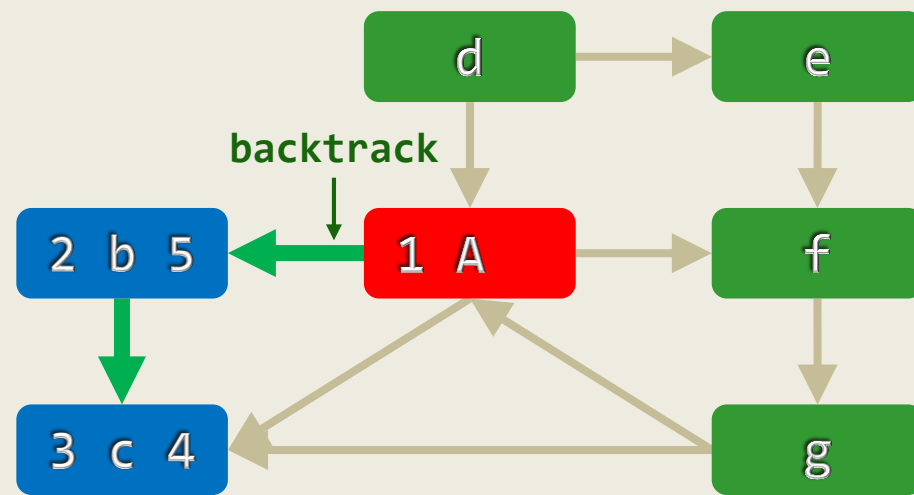
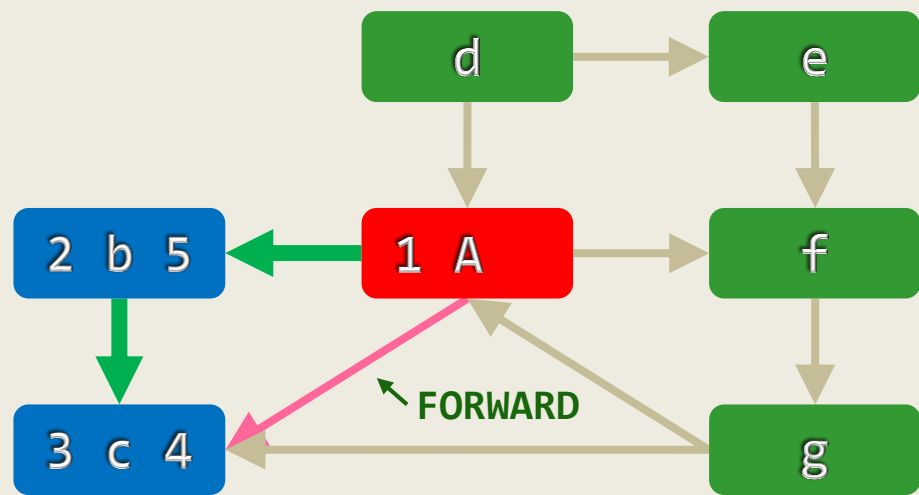
BACKWARD

图

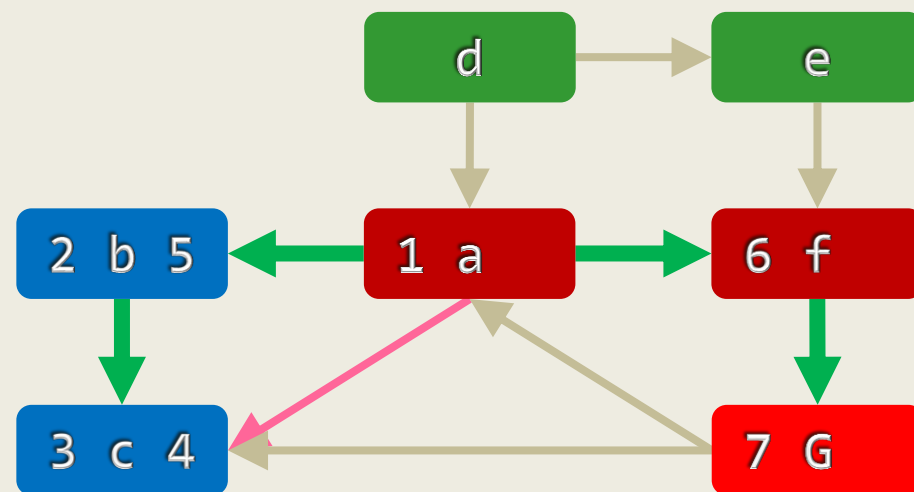
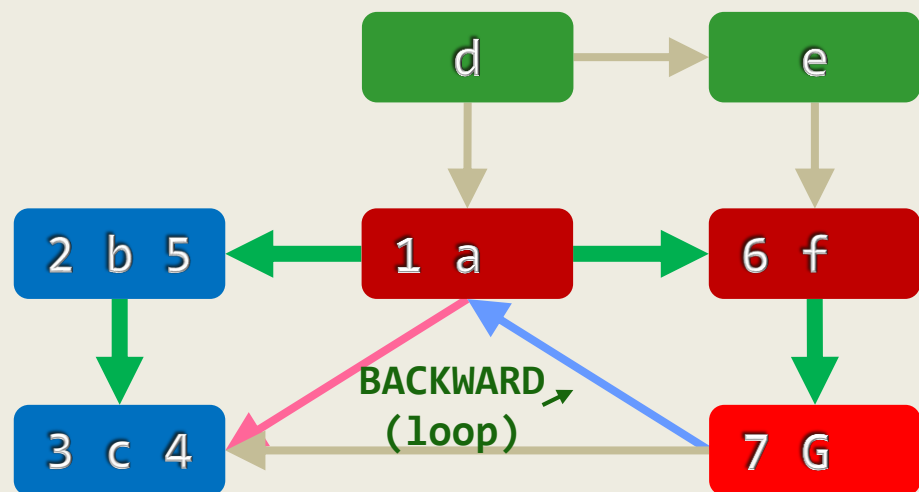
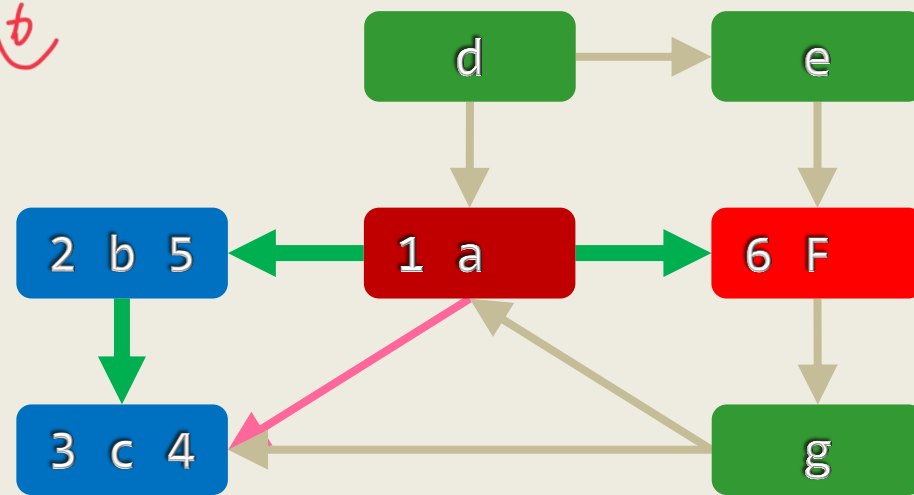
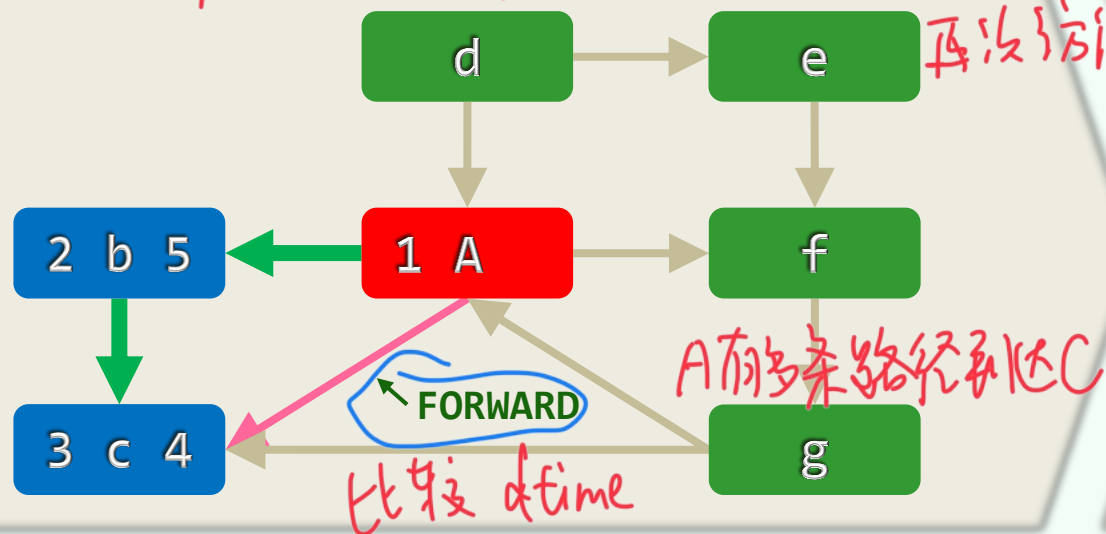
深度优先搜索：实例（有向图）

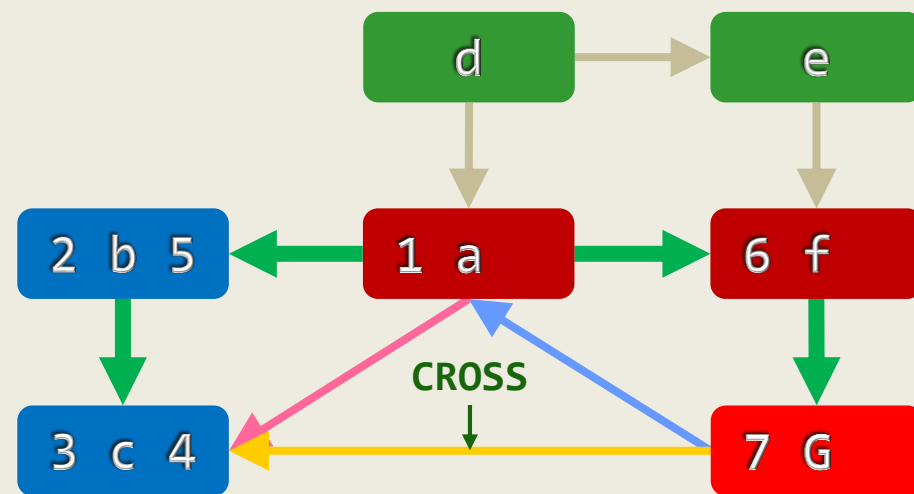
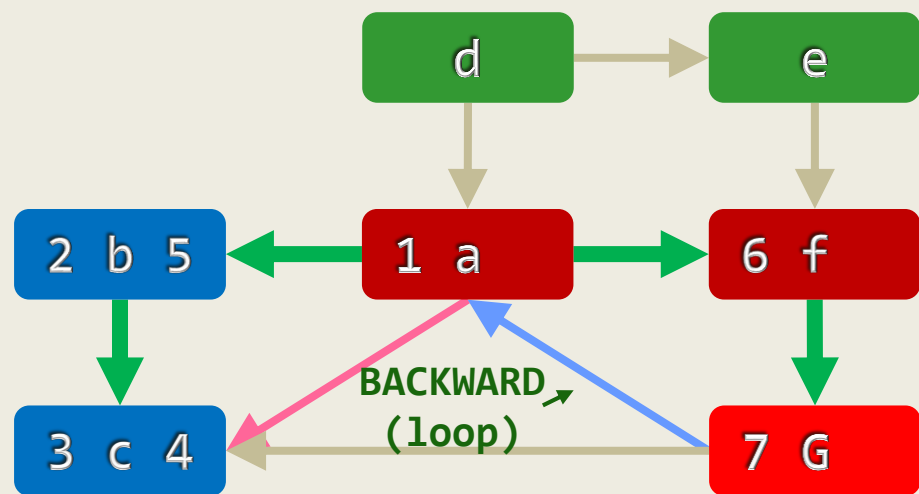
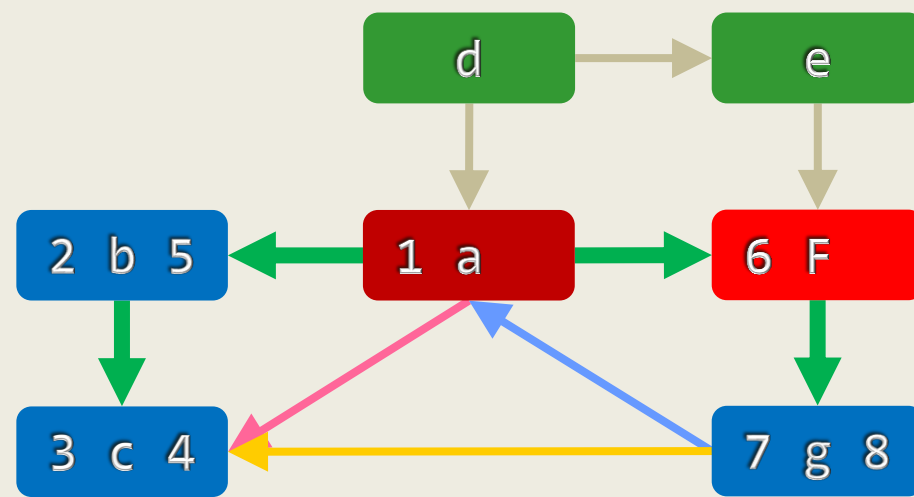
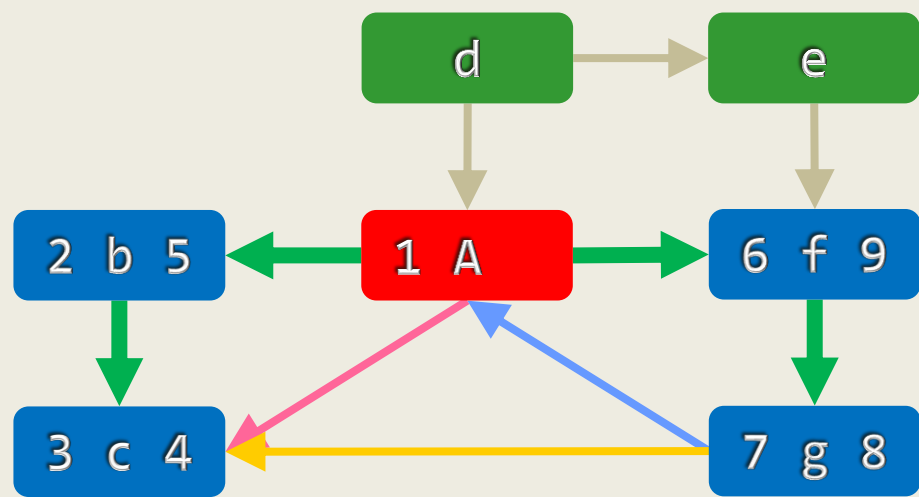
左侧为 dtime, 右侧为 ftime





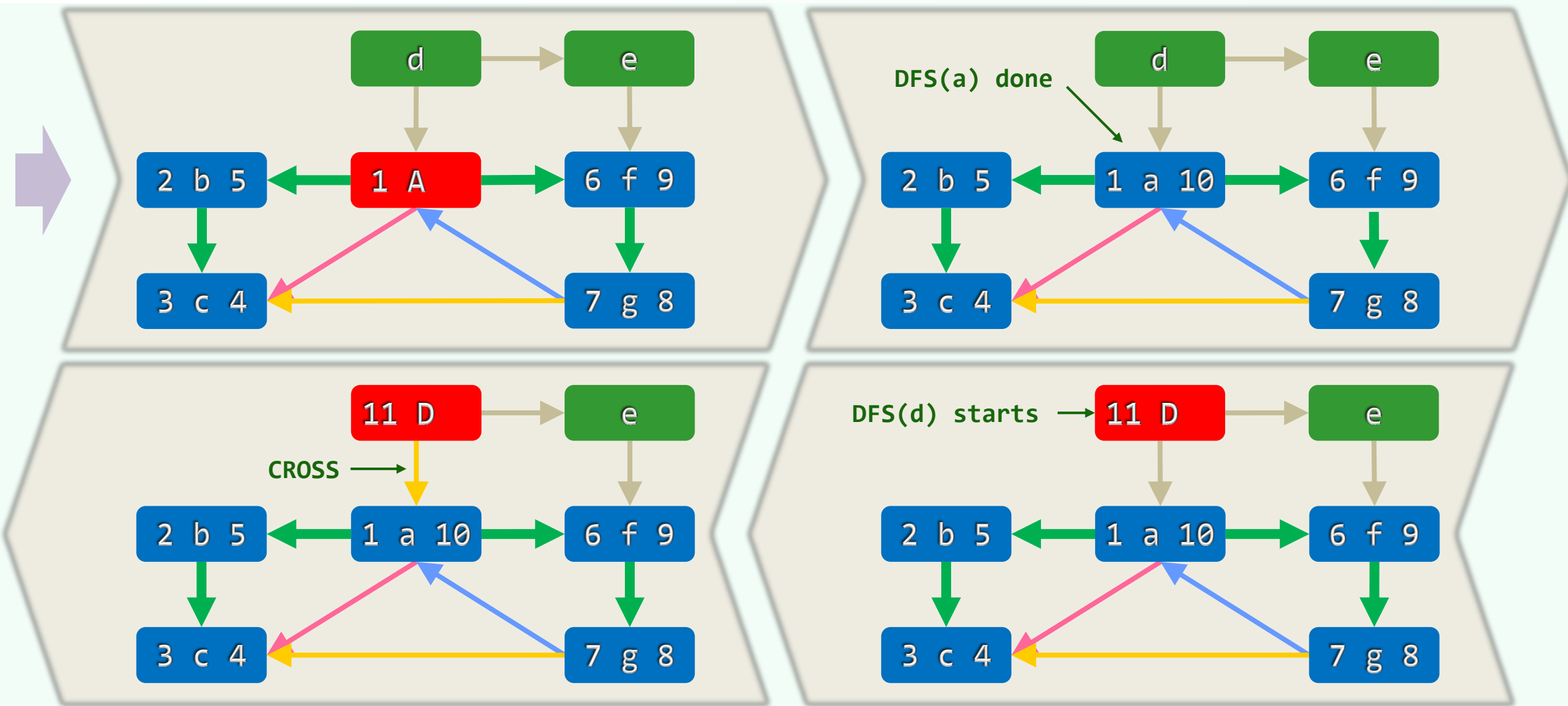
3>1
Forward 前向也. 之前已经到达过?
再没方向的话

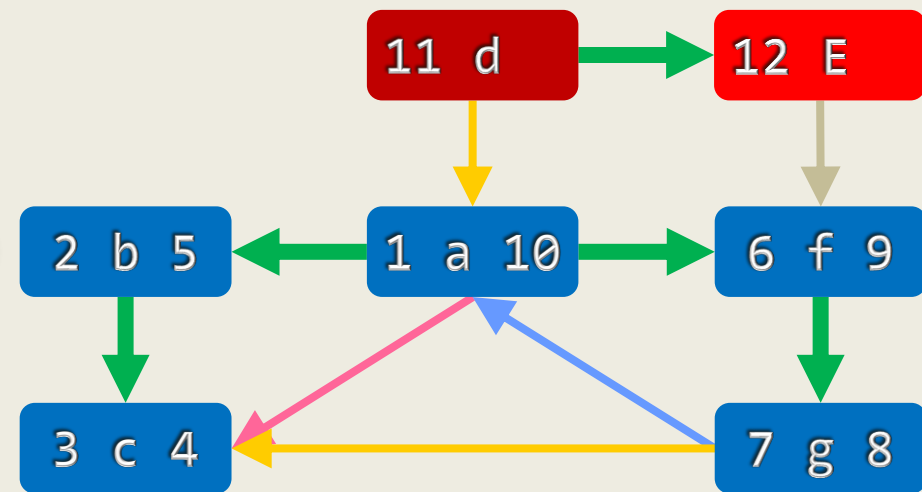
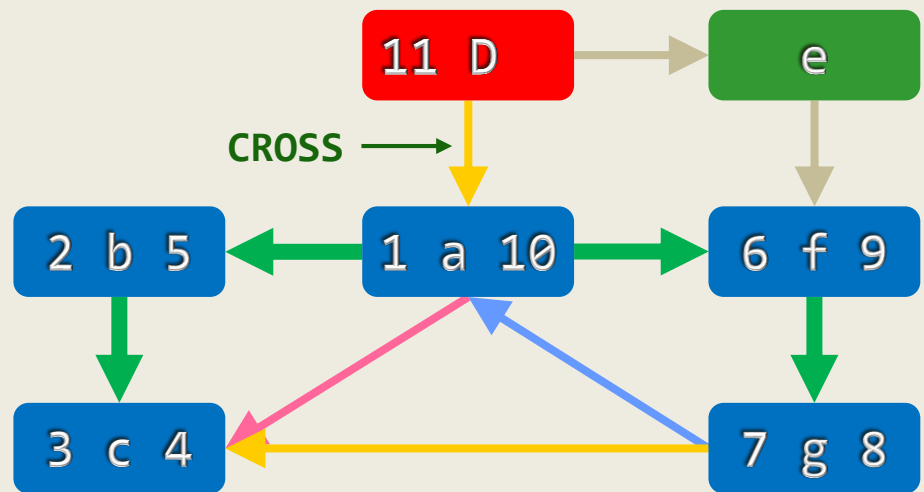
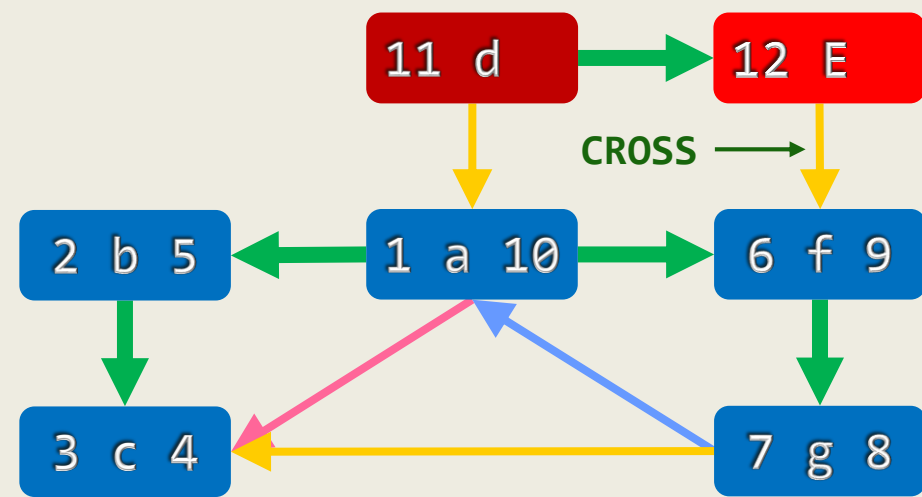
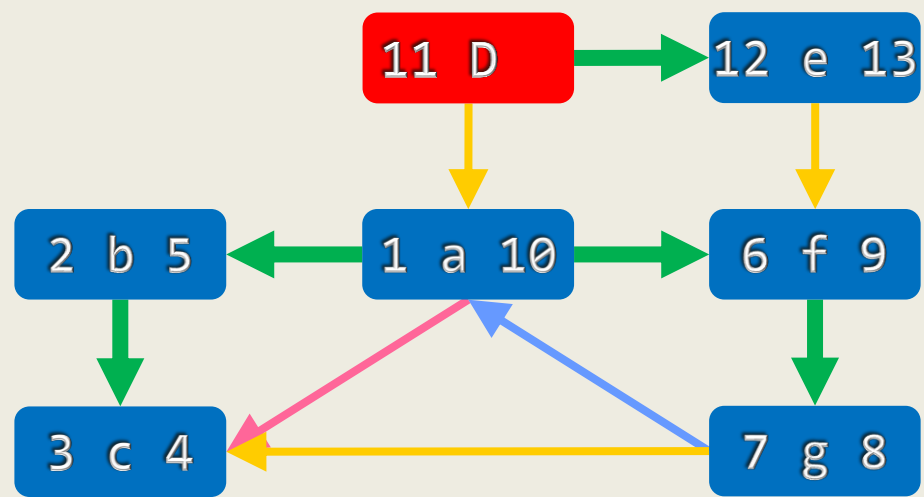


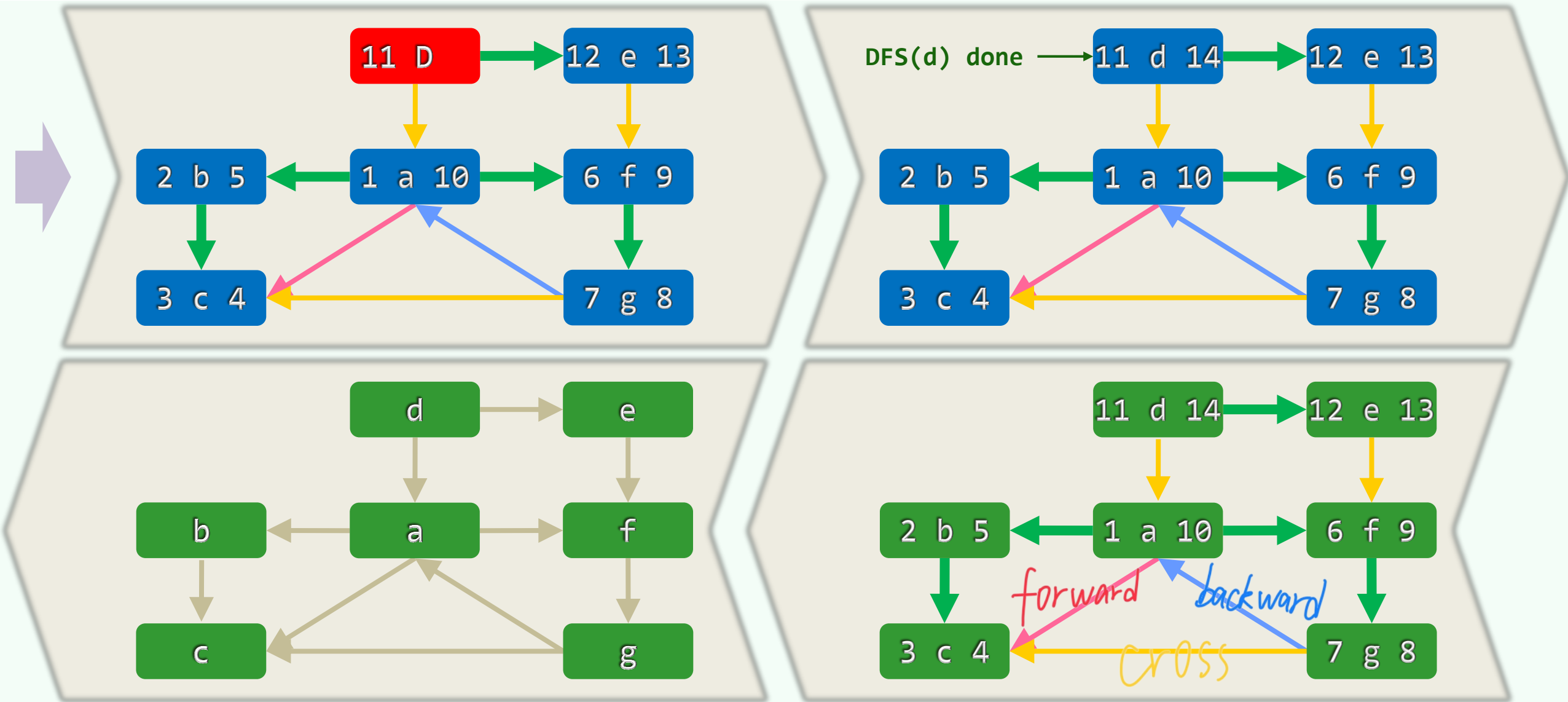


Runtime $3 \leq 7$

Data Structures & Algorithms, Tsinghua University







图

深度优先搜索：性质

括号引理

❖ **活跃期**: $\text{active}[u] = (\text{dTime}[u], \text{fTime}[u])$

❖ **【Parenthesis Lemma】**

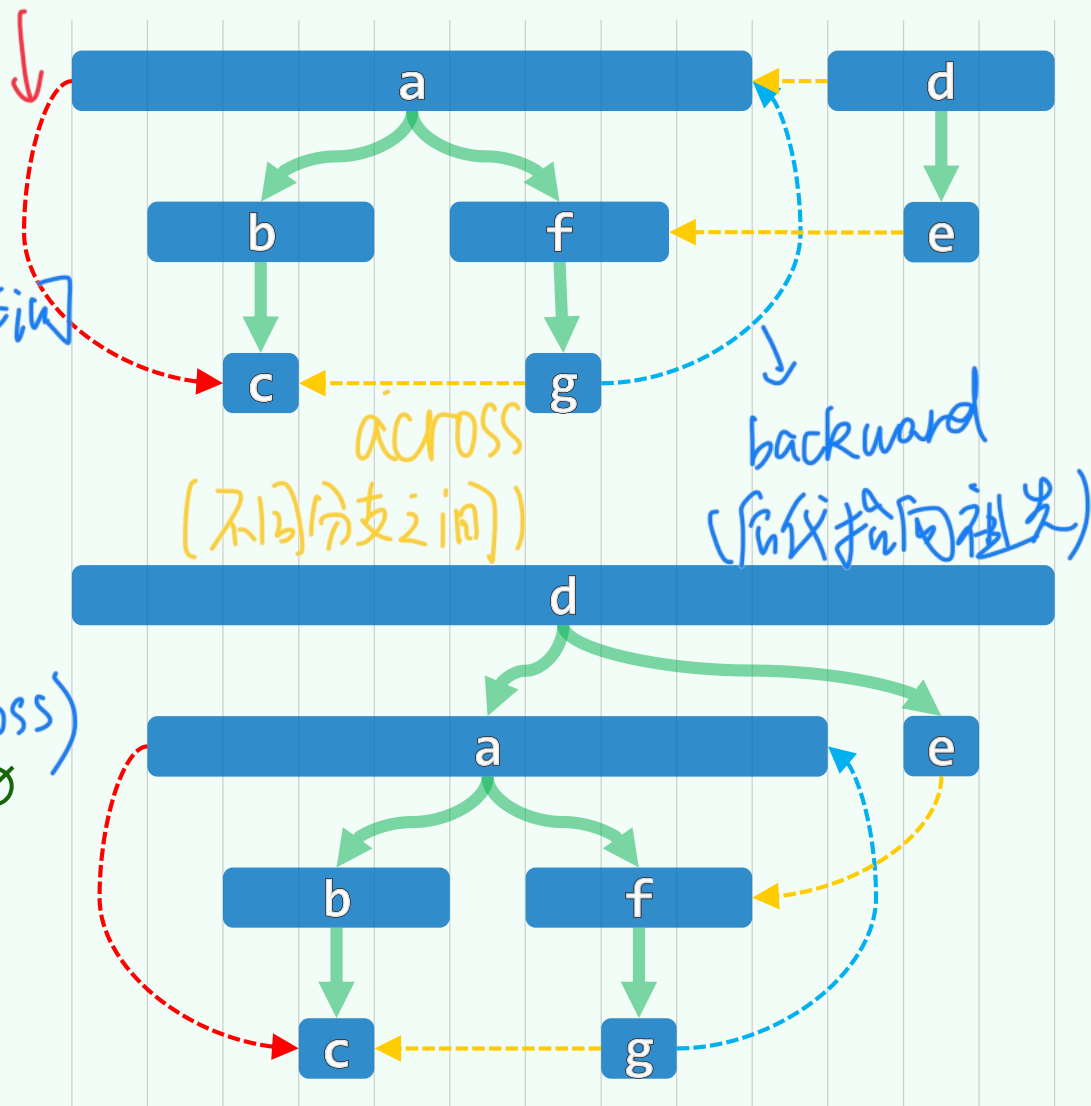
给定有向图 $G = (V, E)$ 及其任一DFS森林, 则

- u 是 v 的后代 iff $\text{active}[u] \subseteq \text{active}[v]$
- u 是 v 的祖先 iff $\text{active}[u] \supseteq \text{active}[v]$
- u 与 v “无关” iff $\text{active}[u] \cap \text{active}[v] = \emptyset$

❖ 仅凭 $\text{status}[]$ 、 $\text{dTime}[]$ 和 $\text{fTime}[]$

即可对各边分类...

forward (祖先指向后代)



图

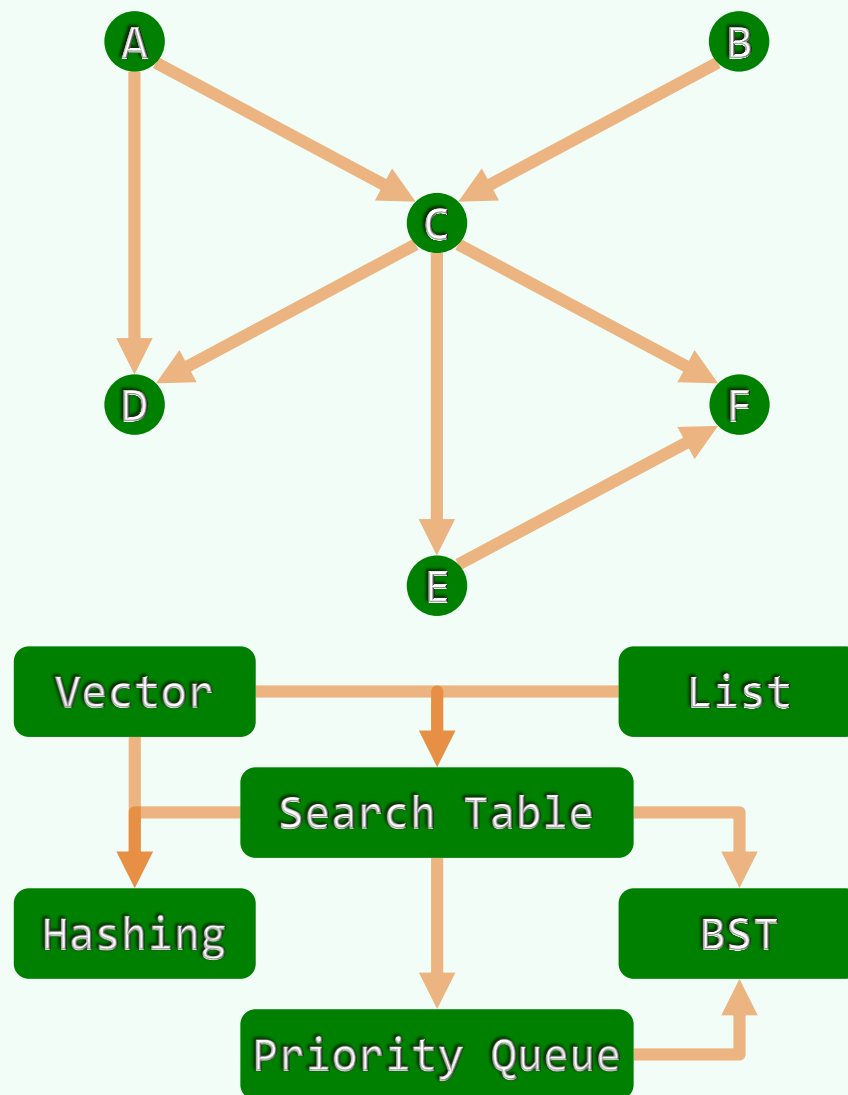
拓扑排序：零入度算法

有向无环图

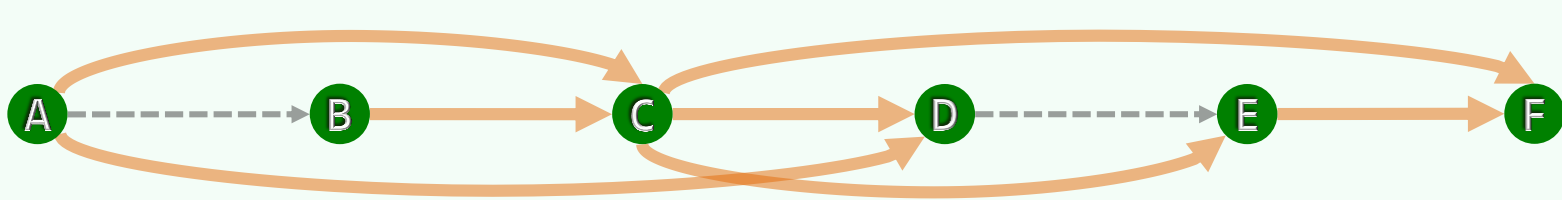
❖ Directed Acyclic Graph

❖ 应用

- 类派生和继承关系图中，是否存在循环定义
- 操作系统中相互等待的一组线程，如何调度
- 给定一组相互依赖的课程，设计可行的培养方案
- 给定一组相互依赖的知识点，设计可行的教学进度方案
- 项目工程图中，设计可串行施工的方案
- email系统中，是否存在自动转发或回复的回路
- ...



拓扑排序



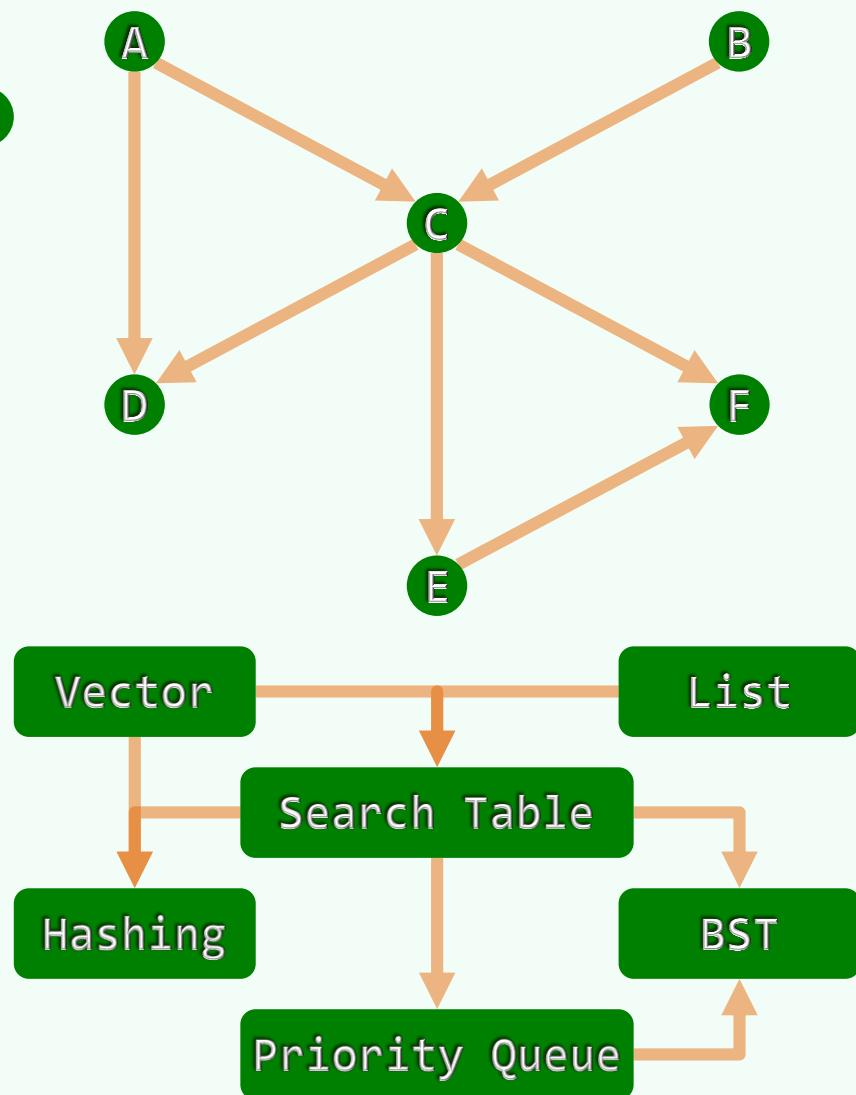
❖ 任给有向图G（不一定是DAG），尝试

将所有顶点排成一个**线性序列**，使其次序须与原图**相容**

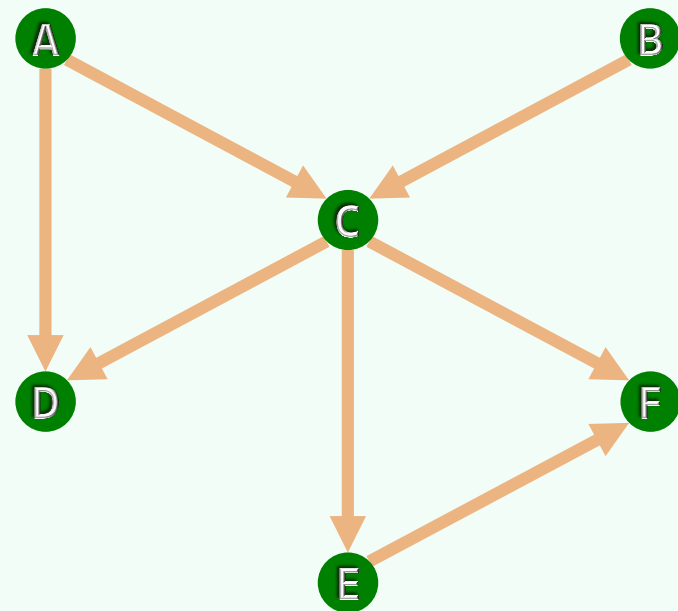
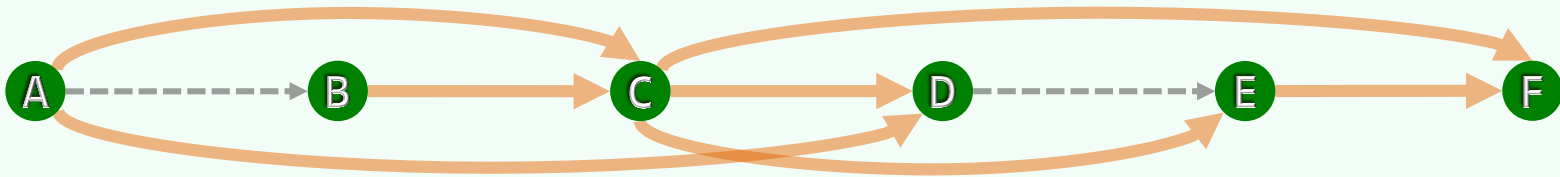
（每一顶点都不会通过边指向**前驱**顶点）

❖ 接口要求

- 若原图存在回路（即并非DAG），检查并报告
- 否则，给出一个相容的线性序列



存在性



❖ 任何有向无环图 $\mathcal{G}$ 中

必有一个零入度的顶点 m

❖ 若 $\mathcal{G} \setminus \{m\}$ 存在拓扑排序 $S = \{ \underline{u_{k_1}, u_{k_2}, u_{k_3}, \dots, u_{k_{n-1}}} \}$ //subtraction

则 $S' = \{ m, \underline{u_{k_1}, u_{k_2}, u_{k_3}, \dots, u_{k_{n-1}}} \}$ 即为 $\mathcal{G}$ 的拓扑排序 //DAG子图亦为DAG

❖ 当然，只要 m 不唯一，拓扑排序也应不唯一 //反之呢？

策略：顺序输出零入度顶点

将所有入度为零的顶点存入栈s，取空队列Q $O(n)$

while (! s.empty()) { $O(n)$

为访问作准备
Q.enqueue(v = s.pop()); // 栈顶v转入队列

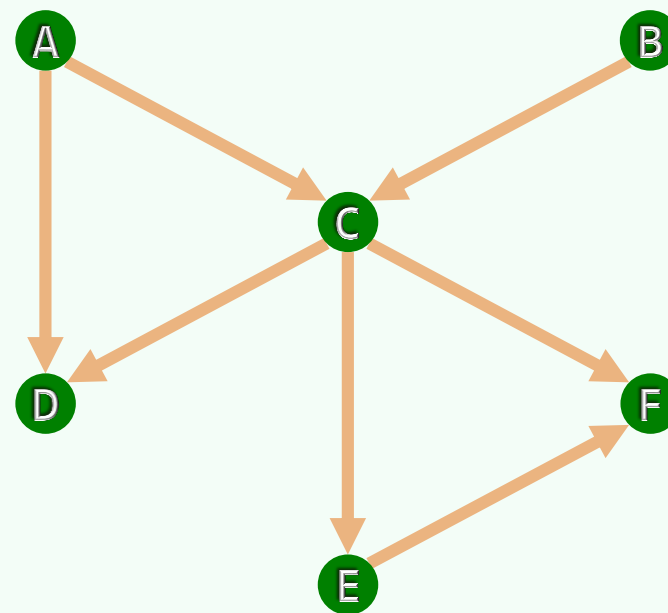
for each edge(v, u) // v的邻接顶点u若入度仅为1

即入度为1
if (u.inDegree < 2) s.push(u); // 则入栈

G = G \ { v }; // 删除v及其关联边 (邻接顶点入度减1)

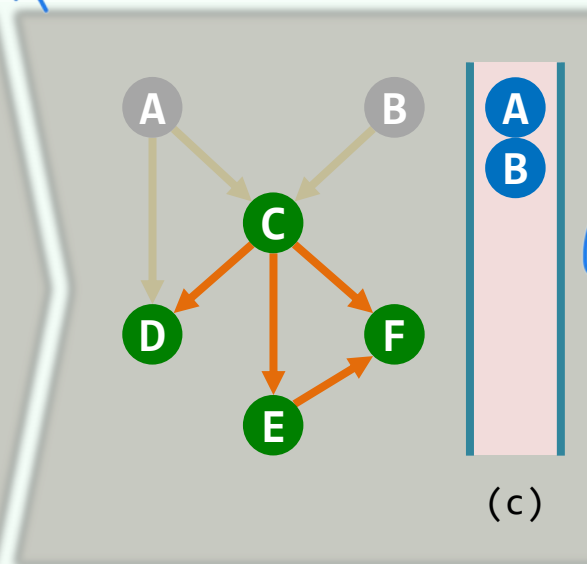
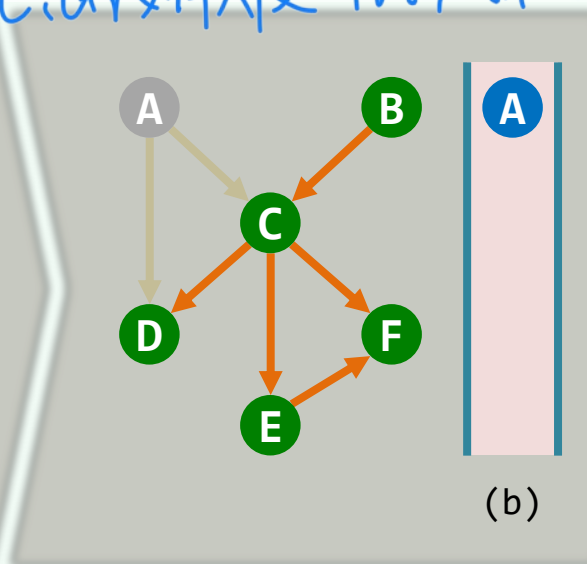
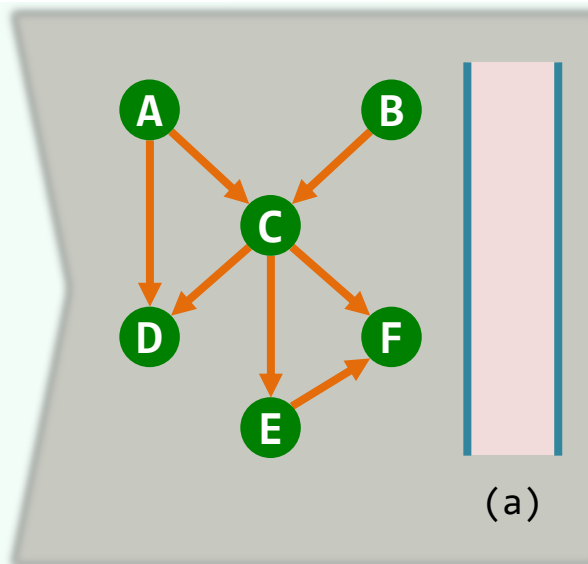
} // 总体 $O(n + e)$

return |G| ? "NOT_A_DAG" : Q; // 残留的G空，当且仅当原图可拓扑排序

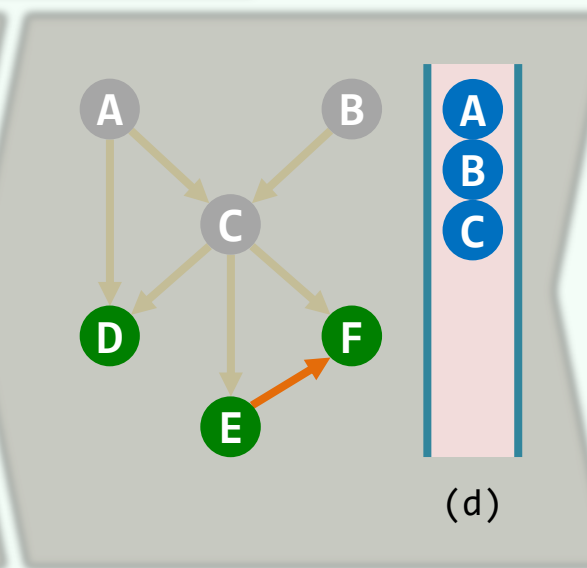
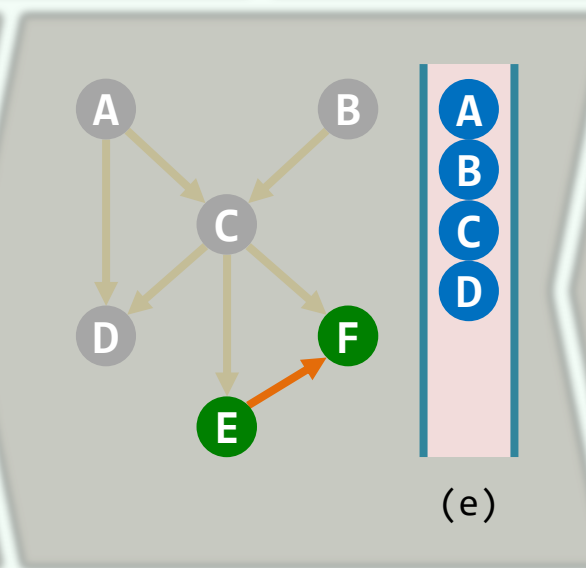
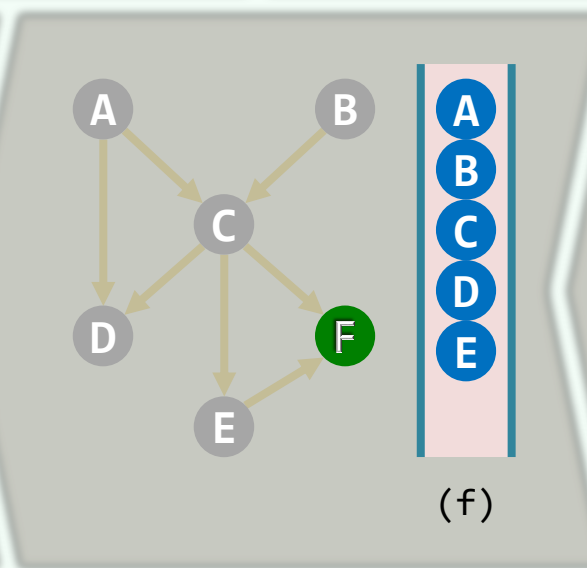
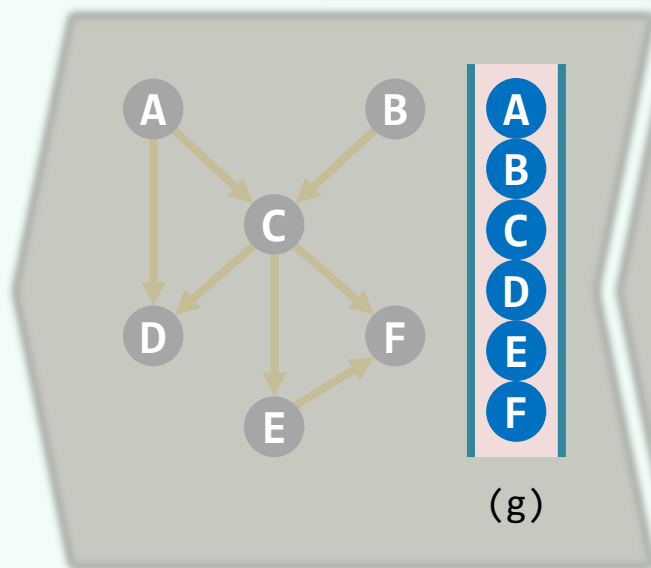


实例

C, d 没有入度=1的, 均不入栈



C此时度为1

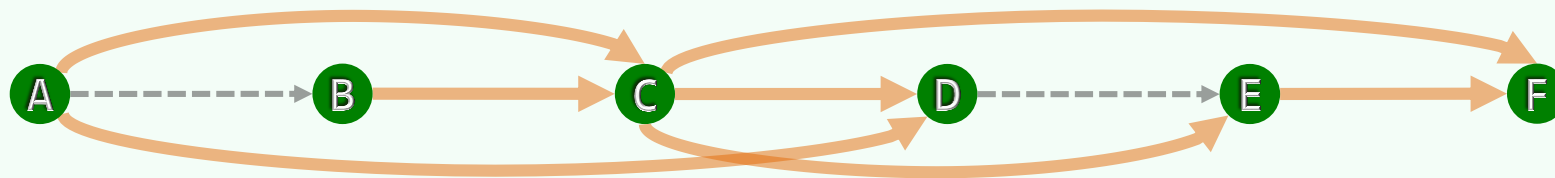


图

拓扑排序：零出度算法

策略：逆序输出零出度顶点

❖ //基于DFS, 借助栈S



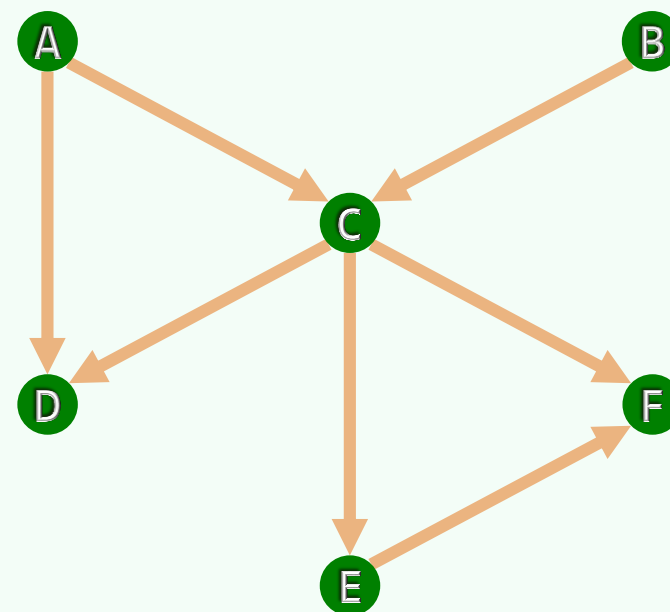
对图G做DFS, 其间 //得到组成DFS森林的一系列DFS树

- 每当有顶点被标记为**VISITED**, 则将其压入S
- 一旦发现有**后向边**, 则报告 "NOT_A_DAG" 并退出
backward → 成环

DFS结束后, 顺序弹出S中的各个顶点

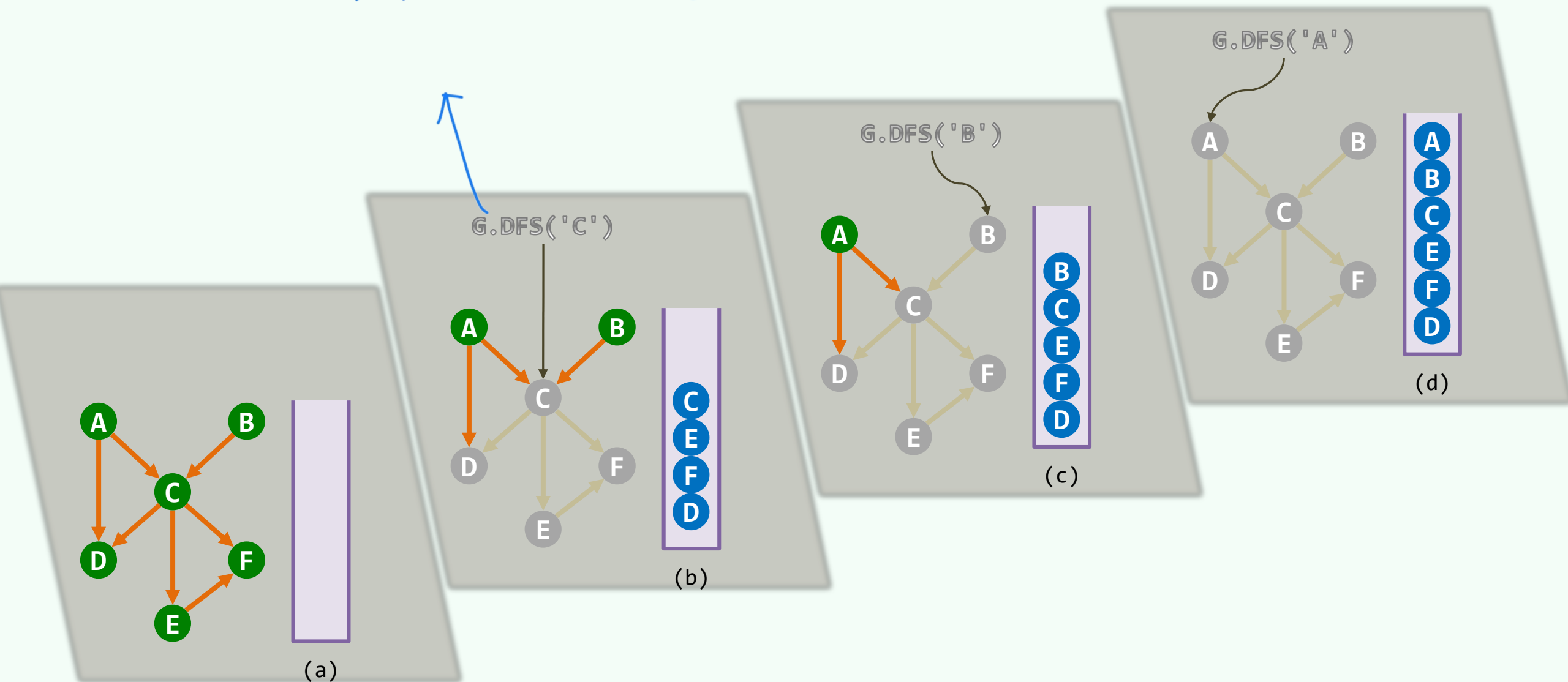
❖ 各节点按**fTime**逆序排列, 即是拓扑排序

❖ 复杂度与DFS相当, 也是 $O(n + e)$



实例

具体顺序 DFE (逆序)



实现 (1/2)

```
template <typename Tv, typename Te> //顶点类型、边类型
bool Graph<Tv, Te>::TSort( Rank v, int & clock, Stack<Tv>* S ) {
    dTime(v) = ++clock; status(v) = DISCOVERED; //发现顶点v 取过它的顶点
    for ( Rank u = firstNbr(v); u < UINT_MAX; u = nextNbr(v, u) ) //考查v的每一邻居u
        /* ... 视u的状态, 分别处理 ... */
    status(v) = VISITED; S->push( vertex(v) ); //顶点被标记为VISITED时入栈
    return true;
}
```

实现 (2/2)

```
for ( Rank u = firstNbr(v); u < UINT_MAX; u = nextNbr(v, u) ) //考查v的每一邻居u
    switch ( status(u) ) { //并视u的状态分别处理
        case UNDISCOVERED:
            parent(u) = v; type(v, u) = TREE;
            if ( ! TSort(u, clock, S) ) return false; break; //从顶点u处深入
            → 第一个 case 已经返回 false
        case DISCOVERED: //一旦发现后向边 (非DAG)
            type(v, u) = BACKWARD; return false; //则退出而不再深入
        default: //VISITED (digraphs only)
            type(v, u) = dTime(v) < dTime(u) ? FORWARD : CROSS; break;
    }
```